

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет _____ Информационных Системы и Технологии _____
Кафедра _____ «Информационные системы» _____
Дисциплина _____ Программирование мобильных устройств _____

КУРСОВОЙ ПРОЕКТ (РАБОТА)

Тема _____ Разработка мобильного приложения «Персонажи вселенной Гарри Поттера» на Flutter для Android _____

Выполнил студент _____ / _____ М.С. Масенькин _____ /
подпись инициалы, фамилия

Курс _____ 3 _____ Группа _____ ПИбд-31 _____

Направление/ специальность _____ Программная инженерия _____

Руководитель _____ доцент, к.т.н., доцент _____
должность, ученая степень, ученое звание
_____ Филиппов Алексей Александрович _____
фамилия, имя, отчество

Дата сдачи:
« _____ » _____ 2024 _ г.

Дата защиты:
« _____ » _____ 2024 _ г.

Оценка: _____

Ульяновск
2024 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет _____ Информационных Системы и Технологии _____
Кафедра _____ «Информационные системы» _____
Дисциплина _____ Программирование мобильных устройств _____

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ (РАБОТУ)

студенту _____ ПИбд-31 _____ М.С. Масенькин _____
группа фамилия, инициалы

Тема проекта (работы) _____ Разработка мобильного приложения «Персонажи
вселенной Гарри Поттера» на Flutter для Android

Срок сдачи законченного проекта (работы) « ____ » _____ 2024 __ г.

Исходные данные к проекту (работе) _____ Задание к курсовой работе _____
(базовое предприятие, характер курсового проекта (работы):

_____ задание кафедры, инициативная НИР, рекомендуемая литература, материалы практики)

Содержание пояснительной записки (перечень подлежащих разработке вопросов)

Перечень графического материала (с точным указанием обязательных чертежей)

Руководитель _____ доцент _____ / _____ А. А. Филиппов _____ /
должность подпись инициалы, фамилия

« ____ » _____ 2024 __ г

Студент _____ / _____ М.С. Масенькин _____ /
подпись инициалы, фамилия

« ____ » _____ 2024 __ г

ОТЗЫВ
руководителя на курсовой проект (работу)

фамилия, имя и отчество

Дисциплина	Программирование мобильных устройств
------------	--------------------------------------

Отмечаются следующие моменты: актуальность темы исследования; соответствие содержания и структуры курсовой работы ее теме; степень разработанности проблемы, наиболее интересно исследованные вопросы. Оценивается степень самостоятельности и инициативы студента; умение пользоваться различными источниками информации; уровень его теоретической подготовки; умение анализировать научные материалы, делать практические выводы; знание основных концепций, научной и специальной литературы по избранной теме. Содержится оценка проекта (работы) руководителем.

« » 2024 г.

Содержание

Содержание	4
Введение.....	5
Проектирование программной системы	8
Информационное обеспечение	8
Интерфейс пользователя.....	11
Архитектура приложения	17
Программный интерфейс и функции серверной части.....	20
Разработка программной системы	23
Слои приложения	23
Слой для работы с хранилищем	26
Слой для реализации бизнес-логики	28
Слой представления	32
Руководство пользователя	37
Заключение	48
Список литературы	51
Исходный код	53

Введение

Мобильное приложение, разработанное в рамках данной курсовой работы, посвящено персонажам вселенной «Гарри Поттера» – одной из самых известных и популярных франшиз в мире. Произведение «Гарри Поттер» является не только литературным феноменом, созданным Дж. К. Роулинг, но и значимой частью мировой поп-культуры, благодаря экранизациям, видеоиграм и театральным постановкам. В 2023 году было объявлено о начале работы над новым телевизионным сериалом по книгам о Гарри Поттере, что делает тему проекта особенно актуальной. Планируемый выход сериала вызвал всплеск интереса к франшизе и её персонажам, что открывает возможности для разработки приложений, которые предоставляют поклонникам доступ к информации и интерактивным функциям, связанным с волшебным миром.

Вселенная «Гарри Поттера» уже несколько десятилетий остаётся в центре внимания множества людей, от детей до взрослых. Фанаты этой вселенной активно ищут способы расширить свои знания о персонажах, событиях и предметах, связанных с волшебным миром. В то же время современные технологии позволяют создавать удобные и интерактивные инструменты для получения и использования этой информации. Мобильные приложения играют важную роль в создании персонализированного опыта для пользователей, позволяя получить доступ к информации в любое время и в любом месте. Приложение, посвящённое персонажам «Гарри Поттера», позволяет не только удовлетворить интересы фанатов, но и поддерживать их вовлечённость в ожидании нового сериала.

Кроме того, наличие функционала, включающего просмотр карточек персонажей, детальную информацию, систему оценок и поддержку локализации, делает приложение полезным для широкой аудитории, включая тех, кто впервые знакомится с этой вселенной. А использование современных технологий, таких как API для получения данных, пагинация и локализация, позволяет продемонстрировать актуальные подходы к разработке программного обеспечения.

Предметная область включает изучение персонажей и событий вселенной «Гарри Поттера», а также автоматизацию процессов, связанных с представлением информации о них. Основными сущностями в данной предметной области являются:

- Персонажи: ключевые герои и второстепенные персонажи, каждый из которых имеет уникальные черты, историю и роль в сюжетной линии.
- Карточки персонажей: краткая информация о герое, включающая имя, описание и изображение.
- Оценки персонажей: возможность пользователей выразить своё отношение к тому или иному герою через лайки и дизлайки.
- Детальная информация: расширенное описание персонажа, включая биографию, ключевые моменты участия в сюжете, а также дополнительные данные, такие как принадлежность к факультету или профессия.

Приложение автоматизирует несколько ключевых бизнес-процессов, связанных с предоставлением информации о персонажах вселенной «Гарри Поттера», их персонализацией и интерактивным взаимодействием с пользователем:

- Отображение списка карточек персонажей: на главном экране приложение автоматически формирует список карточек с краткой информацией о персонажах. Эти карточки отображаются в удобном для восприятия формате. Также реализована подгрузка новых карточек по мере прокрутки списка, что делает интерфейс более динамичным и удобным для пользователя.
- Запрос данных о персонажах из удалённого API: приложение автоматически отправляет запросы к удалённому API для получения актуальной информации о персонажах, включая их имена, описания, изображения и другие атрибуты. Эти данные поступают с учётом поддержки пагинации, что позволяет загружать контент небольшими частями, обеспечивая стабильную

работу приложения и плавный пользовательский опыт. Автоматизация этого процесса устраняет необходимость ручного обновления данных и гарантирует их актуальность.

- Оценка персонажей (лайки/дизлайки): пользователь может выразить своё отношение к персонажу, поставив лайк или дизлайк на карточке. После этого приложение автоматически сохраняет предпочтение в локальном хранилище устройства (shared preferences) и отображает соответствующую отметку при последующих запусках. Этот процесс позволяет создать персонализированный опыт и даёт пользователю возможность влиять на отображение контента.
- Поиск по имени персонажа: приложение предоставляет пользователю возможность быстро находить интересующего персонажа с помощью строки поиска. Процесс поиска полностью автоматизирован: результаты обновляются в реальном времени по мере ввода текста, предлагая релевантные варианты. Это упрощает навигацию по обширному списку персонажей и экономит время пользователя.
- Управление локализацией приложения: интерфейс приложения поддерживает русский и английский языки. Это позволяет облегчить использование приложения для носителей разных языков, обеспечивая поддержку русского и английского языков. Пользователь может вручную выбрать предпочтительный язык в настройках.

Таким образом, разработка мобильного приложения, посвящённого персонажам «Гарри Поттера», является актуальной задачей в условиях растущего интереса к франшизе. Автоматизация процессов, связанных с поиском и предоставлением информации, персонализацией интерфейса и локализацией, делает приложение удобным инструментом для фанатов, а также расширяет их возможности в исследовании волшебного мира.

Проектирование программной системы

Информационное обеспечение

CardData – это основная сущность, описывающая карточку персонажа, отображаемую на главном экране или на детальной странице. Она объединяет в себе текстовое описание, идентификатор карточки и информацию о связанном персонаже.

Атрибуты:

- **id (String?)**: уникальный идентификатор карточки. Используется для определения карточки в базе данных или при взаимодействии с API. Может быть null, если карточка создаётся локально и ещё не синхронизирована с сервером.
- **title (String)**: заголовок карточки, обозначающий тип запрашиваемых данных (в данном случае – «character»). Заголовок всегда отображается на карточке как ключевая информация.
- **text (String)**: текстовое описание карточки. Включает биографические данные, такие как дата рождения и дата смерти.
- **character (CharacterData)**: объект, представляющий персонажа, связанного с данной карточкой. Это поле связывает карточку с конкретной сущностью CharacterData.

Каждая карточка (CardData) связана с одним объектом персонажа (CharacterData). Эта связь типа «один-к-одному» гарантирует, что у каждой карточки есть строго определённый персонаж.

CharacterData описывает данные, непосредственно связанные с персонажем. Это минимальная сущность, которая содержит базовую информацию для отображения.

Атрибуты:

- **name (String)**: имя персонажа, уникально идентифицирующее его в контексте приложения. Это поле обязательно и используется для поиска и отображения подробной информации о персонаже.

- `avatarUrl` (String?): URL-адрес изображения персонажа, используемого для визуализации карточки. Это поле может быть `null`, если изображение персонажа отсутствует. В таком случае приложение может отображать заглушку (силуэт).

`CharacterData` является составной частью `CardData`, так как каждый персонаж представлен одной карточкой.

HomeData служит для управления данными, отображаемыми на главном экране, и содержит список карточек персонажей с поддержкой пагинации.

Атрибуты:

- `data` (List<CardData>?): список объектов `CardData`, представляющих карточки персонажей. Может быть пустым или `null`, если данные ещё не были загружены. Этот список используется для отображения карточек на главном экране с возможностью фильтрации или сортировки.
- `nextPage` (int?): индикатор следующей страницы для загрузки данных. Если это поле `null`, значит данные полностью загружены, и больше страниц не существует.

`HomeData` связана с `CardData` по схеме «один-ко-многим». Один объект `HomeData` содержит список карточек, каждая из которых может быть уникальной или повторяющейся в разных контекстах.

На приведённой ниже ER-диаграмме визуализирована структура данных приложения и связи между его основными сущностями. Диаграмма отражает ключевые объекты (сущности), их атрибуты и логические связи, что упрощает понимание архитектуры данных и логики приложения.

- Сущности и их атрибуты: каждая сущность на диаграмме представлена прямоугольником, внутри которого перечислены её атрибуты. Например, сущности `CardData`, `CharacterData` и `HomeData` содержат атрибуты, подробно описанные выше, такие как `id`, `title`, `name` и другие.

- Связи между сущностями: стрелочки между сущностями обозначают связи. Тип связи указан в виде множественности (например, «один-к-одному» или «один-ко-многим»).
- CardData связана с CharacterData по схеме «один-к-одному», что отображает связь каждой карточки с конкретным персонажем.
- HomeData связана с CardData по схеме "один-ко-многим", так как один объект главного экрана содержит множество карточек.
- Ключевые аспекты связи: диаграмма чётко показывает, какие атрибуты используются для связывания сущностей. Например, связь между CardData и CharacterData реализована через поле character в CardData.

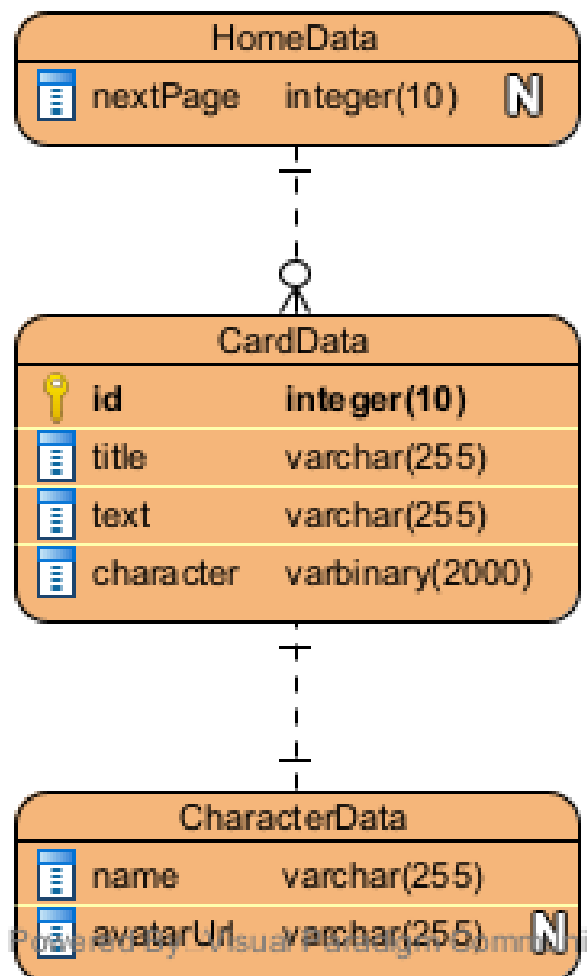


Рисунок 1. ER-диаграмма

Интерфейс пользователя

Главный экран приложения является центральной частью интерфейса, предоставляющей пользователю доступ ко всем основным функциям. Он включает в себя панель AppBar с динамическим цветовым оформлением, которое генерируется случайным образом при каждом запуске приложения. В верхней части отображается название приложения и подпись с указанием разработчика, выполненная в стилизованной форме. Это не только подчёркивает идентичность интерфейса, но и добавляет визуальное разнообразие.

На экране представлена строка поиска, выполненная с использованием компонента CupertinoSearchTextField. Она позволяет пользователю вводить текст для фильтрации списка персонажей по имени. Реализована функция «debounce», которая предотвращает избыточные запросы к API, обрабатывая ввод текста с небольшой задержкой. Это обеспечивает оптимальную производительность при работе с большими данными. Рядом со строкой поиска расположена кнопка переключения языка, представляющая собой SVG-иконки флагов. Нажатие на неё изменяет текущую локализацию интерфейса, а изменение происходит мгновенно, без необходимости перезагрузки приложения.

Основная часть экрана представлена списком карточек персонажей, который отображается с помощью ListView.builder. Каждая карточка содержит имя персонажа, краткое описание и кнопки для взаимодействия (лайк и дизлайк). Карточки адаптированы для работы с состоянием приложения: они выделяются в зависимости от действия пользователя. Состояние взаимодействий, таких как лайки и дизлайки, управляется через ReactionBloc и сохраняется для дальнейшего использования. При прокрутке списка реализована поддержка пагинации: если пользователь достигает конца списка, автоматически загружается следующая страница данных. Это достигается за счёт прослушивания позиции ScrollController.

Дополнительной удобной функцией является поддержка жеста «pull-to-refresh», который позволяет пользователю вручную обновить список, отправив

новый запрос данных. В случае загрузки или ошибок на экране отображаются соответствующие индикаторы: текстовые сообщения об ошибках выделены красным цветом, а процесс загрузки данных сопровождается индикаторами `CircularProgressIndicator`.

Для улучшения пользовательского опыта предусмотрены уведомления в виде `Snackbar`, которые появляются при взаимодействии с кнопками лайков и дизлайков. Уведомления содержат имя персонажа и описание действия (например, «лайк», «дизлайк» или «отмена»), адаптированное к текущей локализации. Это обеспечивает обратную связь и делает интерфейс более интуитивным.

Кроме того, экран поддерживает навигацию к детальной информации о персонаже. При нажатии на карточку открывается экран с подробным описанием, реализованный с использованием `CupertinoPageRoute`. Это упрощает взаимодействие пользователя с большим объёмом данных.

Все элементы интерфейса связаны через архитектуру `BLoC`, что обеспечивает чёткое разделение логики и представления, а также упрощает управление состояниями. Динамическая генерация цвета для `AppBar` и дебаунс-функция для поисковой строки добавляют современности и удобства в работу приложения, а мультиязычность делает его доступным для широкой аудитории. Главный экран ориентирован на интуитивное использование, сохраняя при этом высокую производительность и гибкость в работе с данными.

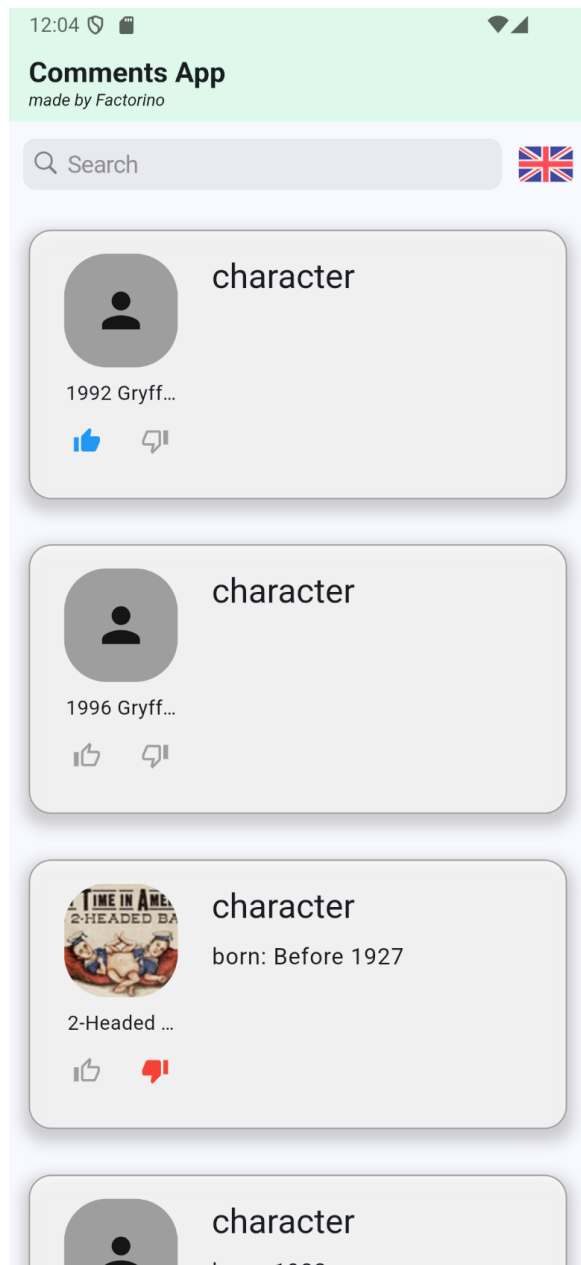


Рисунок 2. Главный экран

Карточка персонажа является ключевым элементом интерфейса, предоставляющим пользователю компактное и информативное представление о каждом персонаже. Она объединяет текстовую и графическую информацию, а также элементы для взаимодействия. Карточка выполнена в минималистичном стиле, но включает всё необходимое для удобной работы пользователя.

Основой карточки является контейнер с закруглёнными углами, декоративной рамкой и тенью, что добавляет объёму и визуальной иерархии элементам на экране. Она адаптируется под разное содержимое и визуально выделяется, улучшая читаемость.

В левой части карточки располагается аватар персонажа. Если URL изображения доступен и валиден, отображается его содержимое с применением Image.network, подстраивающееся под размеры блока 80x80 пикселей. Если изображение недоступно, на месте аватара появляется заглушка в виде серого блока с иконкой пользователя. Аватар оформлен в виде круга за счёт закругления углов с помощью ClipRRect. Под аватаром отображается имя персонажа, обрезаемое многоточием при превышении доступного пространства, иконки для оценки (лайк/дизлайк) расположены ниже.

Функциональность лайков и дизлайков обеспечивается с помощью двух кнопок IconButton, которые отображают соответствующую иконку (закрашенную или контурную) в зависимости от текущего состояния. Лайки и дизлайки окрашены в синий и красный цвет соответственно при активации. При нажатии на кнопки вызываются соответствующие методы _onLike или _onDislike, которые отправляют событие через callback-функции, позволяя сохранять состояние реакции на уровне управления состояниями приложения.

Правая часть карточки содержит текстовую информацию: тип данных – «character» (заголовок карточки) и описание. Заголовок отображается крупным шрифтом, чтобы привлечь внимание, а описание выполнено в стандартном стиле приложения для текста. Компоненты адаптируются к доступному пространству, обеспечивая правильное отображение как длинных, так и коротких текстов.

Карточка поддерживает жест нажатия на саму себя, который инициирует переход на экран с детальной информацией о персонаже. Этот функционал реализован через GestureDetector и передачу callback-функции onTap, что делает карточку интерактивной.

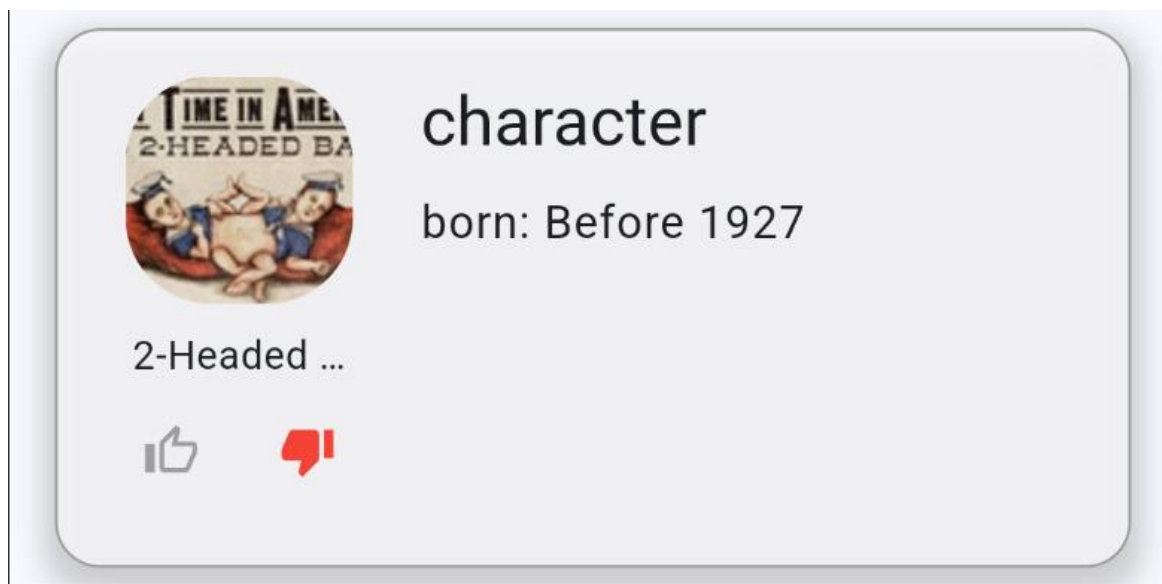


Рисунок 3. Карточка персонажа

Экран с детальной информацией предназначен для отображения расширенных данных о выбранном персонаже, предоставляя пользователю возможность изучить его характеристики в удобном и структурированном формате. Основной структурой экрана является виджет Scaffold, который включает верхнюю панель приложения и основное содержимое. Верхняя панель (AppBar) динамически отображает имя персонажа, что обеспечивает пользователю понимание контекста даже при прокрутке экрана.

В верхней части экрана размещается аватар персонажа, представленный в виде квадратного изображения размером 150x150 пикселей со скругленными углами, созданного с помощью ClipRRect. Если URL изображения доступен, аватар загружается через Image.network и автоматически масштабируется. В случае отсутствия изображения используется заглушка: серый контейнер с иконкой персонажа, что предотвращает появление пустого места. Поверх изображения накладывается контейнер с именем персонажа, выполненный с помощью Stack и расположенный в нижнем левом углу. Для улучшения читаемости имени используется затемнённый фон. Если имя превышает длину в 13 символов, оно автоматически сокращается и дополняется троеточием с помощью вспомогательной функции `_shortenNickname`.

Ниже аватара располагаются текстовые элементы, предоставляющие основную информацию о персонаже. Заголовок, соответствующий названию карточки, выделен крупным жирным шрифтом и оформлен так, чтобы

привлекать внимание пользователя. Описание персонажа размещено под заголовком и выполнено более мелким шрифтом, что делает его удобным для чтения. Расположение текста дополнено равномерными отступами, создающими визуальную гармонию и обеспечивающими комфортное восприятие информации.

Особенностью реализации экрана является его адаптивность: элементы автоматически подстраиваются под размер экрана устройства, сохраняя эстетичный вид. Декоративные элементы, такие как скругление углов и затемнение фона имени, добавляют визуальной привлекательности, а использование заглушек для отсутствующих данных обеспечивает стабильность интерфейса. Дополнительно экран реализован с использованием многоуровневой структуры, позволяющей накладывать элементы друг на друга для создания динамичного дизайна.

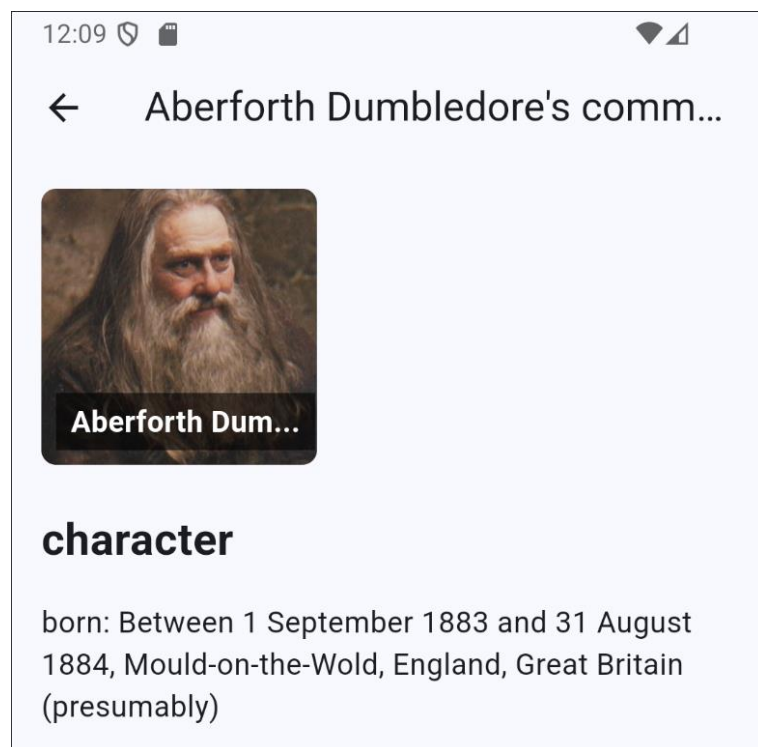


Рисунок 4. Экран детальной информации

Ниже представлена диаграмма переходов между экранами, которая отображает логику навигации внутри приложения. Диаграмма визуализирует основные пути взаимодействия пользователя с приложением, начиная с главного экрана и завершая просмотром детальной информации о персонаже.

Главный экран (HomePage) выступает центральной точкой, с которой пользователь может начинать своё взаимодействие. На этом экране отображается список карточек с персонажами, каждая из которых может быть выбрана для перехода на экран с детальной информацией (DetailsPage). Навигация осуществляется посредством нажатия на карточку, при этом данные о выбранном персонаже передаются на новый экран для отображения.

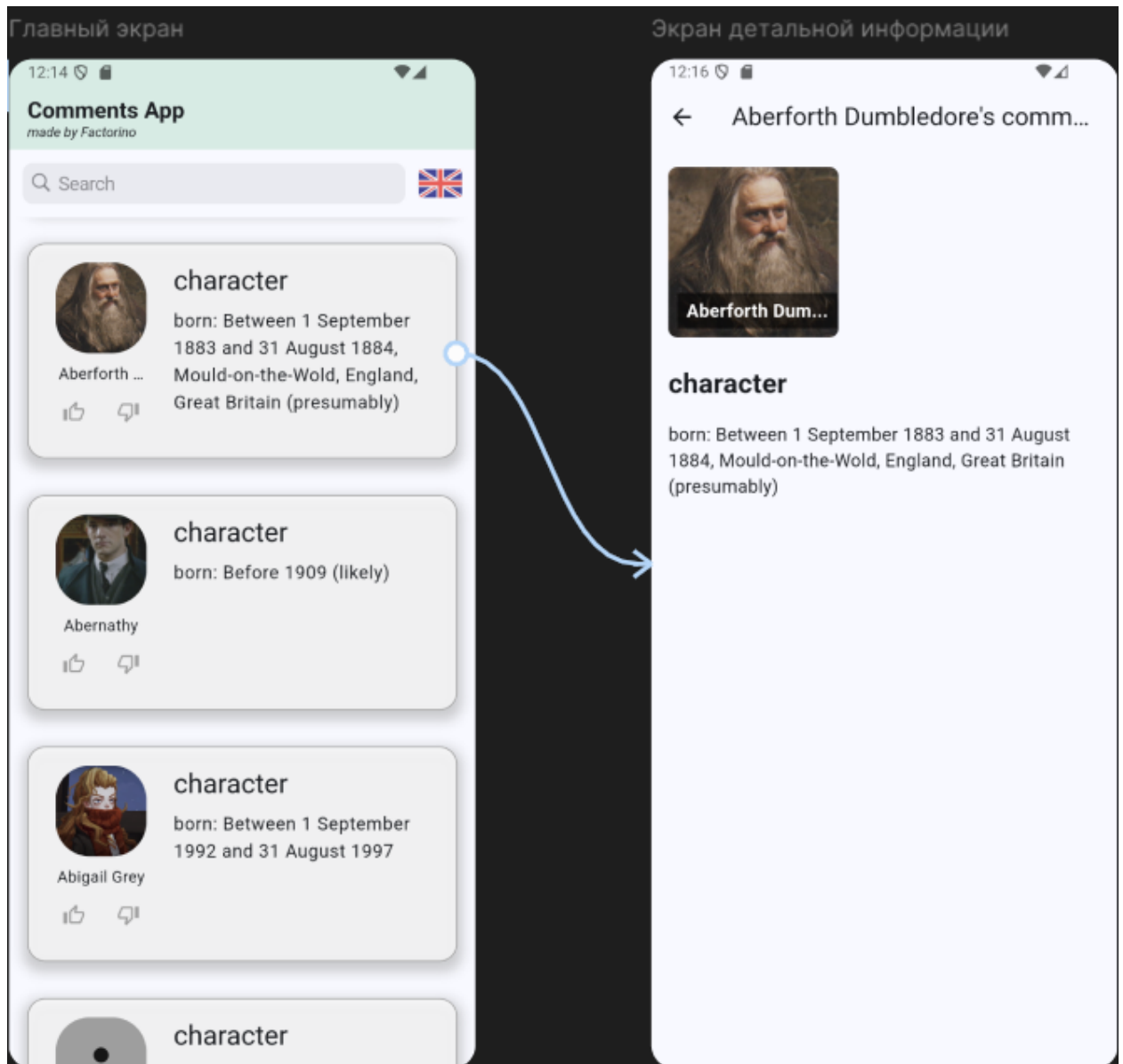


Рисунок 5. Диаграмма переходов

Архитектура приложения

Приложение построено на основе многослойной архитектуры, что обеспечивает четкое разделение ответственности, модульность и удобство сопровождения. Выделяются четыре основных слоя: Components

(Компоненты), Domain (Домен) Data (Данные), и View (Представление). Ниже приведено их описание и назначение.

1. Компоненты (components/): этот слой содержит вспомогательные модули и утилиты, используемые во всём приложении.

- extensions/ – добавляет расширения для удобной работы с контекстами и другими объектами, например, `context_x.dart` предоставляет методы для локализации.
- locale/ – хранит файлы локализации для различных языков (`app_locale_en.dart`, `app_locale_ru.dart`) и их общую конфигурацию.
- utils/ – содержит утилиты, такие как `debounce.dart` для предотвращения избыточного вызова функций.

2. Доменный слой (domain/models/): содержит основные модели данных (`card.dart`, `home.dart`, `character.dart`), которые используются в логике приложения и предоставляют структуру данных.

3. Слой данных (data/): отвечает за взаимодействие с внешними источниками данных, обработку и преобразование данных.

- dtos/ – объекты передачи данных (DTO) для взаимодействия с API.
- mappers/ – преобразователи данных между DTO и внутренними моделями домена.
- repositories/ – реализуют репозитории для работы с различными источниками данных. Например, `mock_repository.dart` для работы с заглушками, `potterdb_repository.dart` для взаимодействия с реальным API.

4. Слой представления (view/): отвечает за отображение интерфейса и взаимодействие с пользователем.

- common/ – общие компоненты, такие как `svg_objects.dart` для отображения SVG.
- details_page/ – реализация экрана с детальной информацией.

- `dialogs/` – универсальные диалоги, например, `error_dialog.dart` для отображения ошибок.
- `home_page/` – экран главной страницы, включает в себя:
 - `bloc/` – реализация логики управления состоянием через BLoC (Bloc, Events, State).
 - UI-компоненты, такие как `card.dart` и основной файл `home_page.dart`.
- `LocaleBloc/` – логика управления состоянием локализации, с использованием BLoC.
- `ReactionBloc/` – управление состоянием реакций (лайки/дизлайки).

Приложение построено на основе архитектурных принципов, обеспечивающих модульность, предсказуемость и удобство сопровождения. Ключевым элементом является использование BLoC (Business Logic Component) для управления состоянием. Этот подход позволяет отделить бизнес-логику от пользовательского интерфейса, делая приложение предсказуемым, тестируемым и легко модифицируемым. Модульная структура обеспечивает изоляцию функциональности, что упрощает работу с кодом, улучшает читаемость и ускоряет внедрение новых функций.

Поддержка мультиязычности реализована через использование локализации, что позволяет адаптировать приложение для различных регионов. Для взаимодействия между слоями данных применяются мапперы, которые преобразуют данные из одного формата в другой, минимизируя зависимость компонентов и повышая их тестируемость.

Такой подход предоставляет множество преимуществ. Разделение ответственности между слоями упрощает поддержку и масштабирование приложения. Модульная структура и изолированная логика делают приложение гибким, что позволяет заменять или модифицировать отдельные компоненты без нарушения работы всей системы. Использование BLoC обеспечивает прозрачность в управлении состоянием и делает приложение

предсказуемым. Кроме того, архитектура приложения упрощает тестирование, позволяя изолированно проверять каждый компонент.

Программный интерфейс и функции серверной части

Potter DB API – это RESTful интерфейс, предоставляющий доступ к структурированным данным из вселенной «Гарри Поттера». Он реализован в соответствии со спецификацией JSON:API, что обеспечивает стандартизацию запросов и ответов, упрощая их обработку и интеграцию. Данный API предоставляет возможность работы с различными ресурсами, такими как персонажи, книги, фильмы, зелья и заклинания, позволяя разработчикам сосредоточиться на создании приложений, используя готовые данные.

Ключевые особенности API:

- Структура данных: каждый ресурс представлен объектом с обязательными атрибутами `type`, `id` и `attributes`, обеспечивая консистентность и легкость в обработке данных.
- Связи (Relationships): ресурсы могут быть связаны друг с другом. Например, книги связаны с главами, а персонажи – с домами.
- Пагинация: ответы с большим количеством данных автоматически разбиваются на страницы. Для навигации предоставляются ссылки на первую, последнюю, предыдущую и следующую страницы. Параметр `page[size]` позволяет настроить количество записей на странице (максимум – 100).
- Обработка ошибок: все ошибки возвращаются в стандартизированном формате JSON, что упрощает их анализ и обработку.

API предоставляет множество эндпоинтов, но приложение использует только эндпоинт для получения данных о персонажах.

Персонажи:

- `/v1/characters`: получение списка персонажей.

- `/v1/characters/{id}`: получение подробной информации о конкретном персонаже по его ID.

Прочие эндпоинты (в приложении не используются):

- Книги: информация о книгах и их главах.
- Фильмы: данные о фильмах.
- Зелья: сведения о магических зельях.
- Заклинания: список заклинаний с подробными характеристиками.

В приложении используется только эндпоинт для работы с персонажами, что позволяет загружать их данные для отображения и дальнейшей обработки. Для этого применяется интерфейс `ApiInterface`, который абстрагирует взаимодействие с сервером и обеспечивает модульность. Интерфейс включает метод `loadData`, который обращается к API для получения списка персонажей и поддерживает обработку ошибок. Это позволяет легко изменять источник данных (например, заменять реальный API на заглушку) без изменения основной логики приложения.

DTO (Data Transfer Objects) в данном приложении используются для обработки и передачи данных, полученных от API, в строго типизированной форме. Они обеспечивают удобное взаимодействие с данными, минимизируют вероятность ошибок и упрощают преобразование данных между сервером и клиентом.

Особенности реализации:

- Аннотация `@JsonSerializable`: DTO используют аннотацию из пакета `json_annotation`, что позволяет автоматически генерировать код для сериализации и десериализации JSON с помощью `build_runner`. Опция `createToJson: false` исключает необходимость обратного преобразования в JSON, поскольку данные используются только для чтения.
- Согласованность со структурой API: DTO отражают структуру JSON-ответов API, включая вложенные объекты для сложных данных, что упрощает их преобразование и обработку.

- Генерация кода: код для преобразования JSON в объекты Dart автоматически генерируется, что снижает рутину и уменьшает вероятность ошибок при работе с данными.
- Строгая типизация и модульность: использование строго типизированных объектов обеспечивает надежность работы приложения и упрощает поддержку и тестирование кода.

Структура DTO:

1. **CharactersDto** – корневой объект, содержащий основные данные о персонажах и метаданные:
 - data – массив объектов типа **CharacterDataDto**, содержащих данные о персонажах.
 - meta – объект **MetaDto** с информацией о запросе, например о пагинации.
2. **CharacterDataDto** – представляет данные о каждом персонаже:
 - id – уникальный идентификатор персонажа.
 - type – тип ресурса, например, «character».
 - attributes – объект **CharacterAttributesDataDto**, содержащий детальную информацию.
3. **CharacterAttributesDataDto** – описывает атрибуты персонажа:
 - name – имя персонажа.
 - born – дата рождения.
 - died – дата смерти.
 - image – URL изображения персонажа.
4. **MetaDto** – обеспечивает дополнительную информацию о запросе:
 - pagination – объект **PaginationDto**, содержащий детали пагинации.
5. **PaginationDto** – описывает параметры пагинации:
 - current – номер текущей страницы.
 - next – номер следующей страницы (если доступна).
 - last – номер последней страницы.

Разработка программной системы

Слои приложения

Приложение построено на многослойной архитектуре, что способствует четкому разделению ответственности, модульности и удобству сопровождения. Каждый слой выполняет свою специфическую роль и взаимодействует с другими слоями через четко определенные интерфейсы.

Слой компонентов (Components): этот слой содержит вспомогательные модули и утилиты, которые используются во всей системе. Он включает:

- `extensions/`: расширения для удобной работы с различными объектами, такими как контексты и другие элементы. Пример: `context_x.dart` содержит методы для локализации, улучшая взаимодействие с контекстом.
- `locale/`: файлы локализации для различных языков, такие как `app_locale_en.dart` для английского и `app_locale_ru.dart` для русского языков. Эти файлы содержат настройки и переводы, которые позволяют адаптировать приложение для разных регионов.
- `utils/`: вспомогательные утилиты, например, `debounce.dart`, который предотвращает избыточные вызовы функций, улучшая производительность.

Доменный слой (Domain): в этом слое находятся основные модели данных, которые используются в бизнес-логике приложения. Он обеспечивает структуру данных и их использование в логике работы приложения.

- `models/`: модели данных, такие как `card.dart`, `home.dart`, и `character.dart`, которые представляют сущности, используемые в приложении.

Слой данных (Data): этот слой отвечает за взаимодействие с внешними источниками данных (например, API) и их обработку. Он включает:

- dtos/: объекты передачи данных (DTO), такие как `characters_dto.dart`, которые представляют данные, передаваемые между слоем данных и другими слоями.
- mappers/: классы для преобразования данных между различными форматами. Например, `characters_mapper.dart` преобразует данные из формата DTO в внутренние модели домена.
- repositories/: репозитории, которые абстрагируют доступ к данным, предоставляя удобные методы для работы с различными источниками данных, например, `mock_repository.dart` для работы с заглушками и `potterdb_repository.dart` для взаимодействия с реальным API.

Слой представления (View): этот слой отвечает за отображение интерфейса и взаимодействие с пользователем. Он включает в себя компоненты, экраны и логику управления состоянием:

- common/: общие компоненты, такие как `svg_objects.dart`, который отображает SVG-графику.
- details_page/: экран с детальной информацией о сущности, реализованный в `details_page.dart`.
- dialogs/: универсальные диалоги, например, `error_dialog.dart` для отображения ошибок.
- home_page/: главный экран, который включает:
 - bloc/: реализация логики управления состоянием через BLoC. Включает файлы, такие как `bloc.dart`, `events.dart`, и `state.dart`, которые управляют состоянием главного экрана.
 - UI-компоненты, такие как `card.dart` и главный файл `home_page.dart`.
- LocaleBloc/: логика управления состоянием локализации с использованием BLoC. Включает файлы `LocaleBloc.dart`, `LocaleEvents.dart` и `LocaleState.dart`, которые обеспечивают переключение локализаций.

- **ReactionBloc/**: управление состоянием реакций (например, лайков и дизлайков). Включает файлы `ReactionBloc.dart`, `ReactionEvents.dart` и `ReactionState.dart`.

Модульная структура приложения предполагает, что каждый слой взаимодействует с другими слоями через интерфейсы и абстракции. Взаимодействие между слоями организовано следующим образом:

- **Слой представления (View)** взаимодействует с доменным слоем (**Domain**) для получения и отображения данных. Блоки управления состоянием, такие как `LocaleBloc`, `ReactionBloc` и другие, управляют состоянием и передают его в интерфейс.
- **Слой данных (Data)** взаимодействует с доменным слоем (**Domain**) через мапперы, которые преобразуют данные из DTO в модели, используемые в бизнес-логике.
- **Слой компонентов (Components)** предоставляет утилиты и расширения, которые облегчают взаимодействие между различными слоями, например, для работы с контекстами, локализацией и другими вспомогательными функциями.

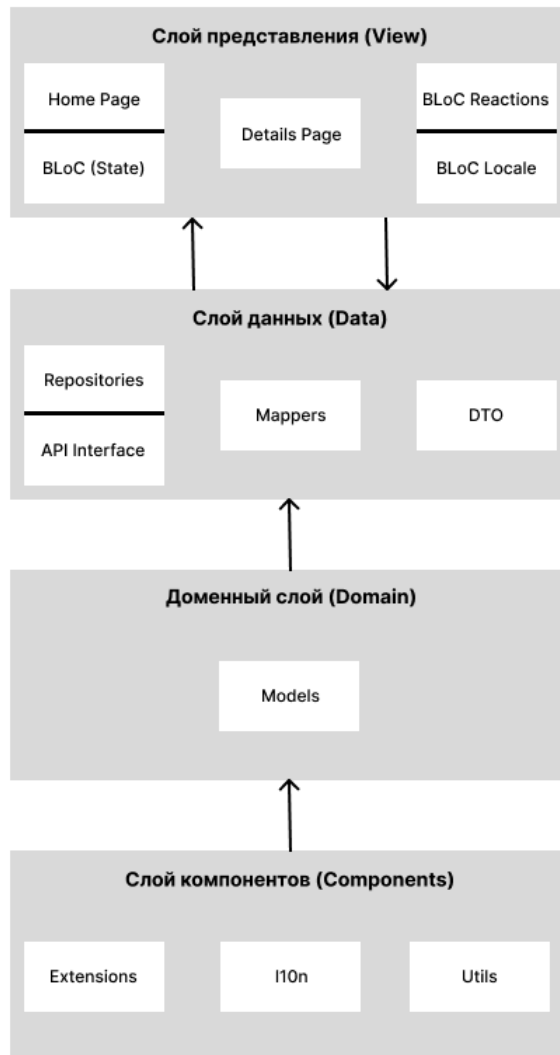


Рисунок 6 Схема взаимодействия между слоями

Слой для работы с хранилищем

Слой для работы с хранилищем отвечает за получение данных из внешних источников, их обработку и предоставление другим слоям приложения в виде унифицированных доменных моделей. Основная задача слоя – абстрагировать детали работы с данными и обеспечить гибкость их получения. Он состоит из интерфейса `ApiInterface` и двух реализаций: тестового репозитория `MockRepository` и репозитория для взаимодействия с реальным API – `PotterDBRepository`.

`ApiInterface` является базовым контрактом для всех репозиториев. Он определяет метод `loadData`, который асинхронно загружает данные и возвращает объект `HomeData`, содержащий обработанную информацию. Метод также поддерживает передачу колбэка `onError` для обработки ошибок.

MockRepository используется для тестирования и разработки. Он предоставляет заранее подготовленный набор данных в виде объекта HomeData, содержащего список тестовых карточек. Каждая карточка включает заголовок (title), текст (text) и информацию о персонаже (CharacterData), включающую имя (name) и ссылку на аватар (avatarUrl). Репозиторий не выполняет сетевые запросы, а данные генерируются локально, что исключает вероятность ошибок, связанных с сетью.

PotterDBRepository реализует взаимодействие с API Potter DB для получения реальных данных о персонажах. Он использует библиотеку Dio для выполнения HTTP-запросов и включает PrettyDioLogger для логирования запросов и ответов, что облегчает отладку. Метод loadData репозитория отправляет GET-запрос к эндпоинту /v1/characters, поддерживает фильтрацию данных по имени персонажа (q) и параметры пагинации (номер страницы page и количество элементов pageSize). Ответ API преобразуется с использованием DTO (CharactersDto) и мапперов в доменные модели (HomeData). Обработка сетевых ошибок реализована через DioException, а сообщение об ошибке передается через onError.

Каждый репозиторий возвращает объект HomeData, который содержит список карточек (CardData). Карточки включают заголовок, текст и данные о персонаже, описанные объектом CharacterData. Такой подход обеспечивает модульность и тестируемость слоя данных, а также гибкость в замене источников данных – от тестового хранилища до реального API.

Маппер в слое работы с хранилищем предназначен для преобразования объектов DTO (Data Transfer Object), полученных от API, в доменные модели, используемые в бизнес-логике приложения. Это обеспечивает изоляцию логики обработки данных от структуры, заданной внешним API, и позволяет унифицировать данные для работы на уровне приложения.

Мапперы реализованы в виде расширений (extension) для классов DTO, что делает их интеграцию и использование лаконичными и удобными. Основным маппером, CharactersDtoToModel, преобразует объект CharactersDto в объект HomeData. Поле data из DTO преобразуется в список объектов CardData

через вызов метода `toDomain()` для каждого элемента списка. Поле `nextPage` берется из информации о пагинации в объекте `meta`. Это упрощает представление данных, полученных от API, для последующего отображения в приложении.

Второй маппер, `CharacterDataDtoToModel`, выполняет преобразование отдельных объектов `CharacterDataDto` в объекты `CardData`. Поле `id` передается напрямую, а поле `type` используется в качестве заголовка (`title`) с указанием значения по умолчанию "UNKNOWN" при отсутствии данных. Поле `attributes` используется для формирования текста описания (`text`), который генерируется с помощью метода `_makeDescriptionText`. Этот метод создает строку, представляющую даты рождения и смерти персонажа: если доступны обе даты, формируется диапазон; если доступна только одна дата, указывается ее тип (рождение или смерть). При отсутствии обеих дат возвращается пустая строка. Также из `attributes` извлекаются имя и URL аватара персонажа, которые передаются в поле `character`.

Мапперы обеспечивают устойчивость к неполным данным, гарантируя, что приложение будет корректно работать даже при отсутствии некоторых полей в ответе API. Реализация в виде расширений логически отделяет маппинг от DTO, повышая читаемость и упрощая поддержку кода. Если структура API или доменной модели изменится, потребуется обновить только соответствующие мапперы, что делает приложение гибким и легко модифицируемым.

Слой для реализации бизнес-логики

Слой бизнес-логики для работы с главной страницей приложения реализован с использованием паттерна BLoC и обеспечивает четкое разделение бизнес-логики и представления. Он состоит из трех ключевых компонентов: состояния, событий и самого блока `HomeBloc`.

Класс `HomeState` представляет текущее состояние главной страницы. Оно включает следующие поля:

- `data` типа `HomeData?`, содержащее данные для отображения.
- `isLoading` – булевый флаг, указывающий на процесс загрузки данных.
- `isPaginationLoading` – булевый флаг, сигнализирующий о подгрузке следующих страниц.
- `error` – строка, содержащая текст ошибки, если загрузка завершилась неудачно. Состояние поддерживает сравнение с помощью пакета `Equatable` для корректной работы `BLoC`. Автоматически сгенерированный метод `copyWith`, созданный с помощью пакета `copy_with_extension`, позволяет копировать состояние с изменением только необходимых полей, что облегчает управление состояниями в блоке.

Класс `HomeEvent` является базовым для всех событий, изменяющих состояние главной страницы. Производный класс `HomeLoadDataEvent` используется для инициации загрузки данных. Он содержит параметры `search` для передачи текста поиска и `nextPage` для указания номера следующей страницы, если требуется подгрузка.

Класс `HomeBloc` реализует основную бизнес-логику. Его конструктор принимает объект `potterdb_repository`, который инкапсулирует взаимодействие с API, и устанавливает начальное состояние с помощью `HomeState`. Обработка событий выполняется методом `on<HomeLoadDataEvent>`, который обрабатывает загрузку данных. Если загружается первая страница, устанавливается состояние `isLoading = true`, для последующих страниц включается флаг `isPaginationLoading = true`. Затем выполняется вызов метода `loadData` из репозитория, который загружает данные и обрабатывает возможные ошибки. Если загружаются данные для последующих страниц, они объединяются с уже существующими. После завершения загрузки состояние обновляется, включая флаги загрузки, загруженные данные и текст ошибки (если она произошла).

Особенностями реализации слоя являются четкое разделение ответственности между событиями, состояниями и логикой в блоке, удобное

управление состояниями через `copyWith`, автоматическая генерация вспомогательного кода и использование пакета `Equatable` для сравнения объектов.

Слой бизнес-логики для управления локализацией приложения также реализован с использованием `BLoC` и включает три ключевых компонента: состояния, события и блок `LocaleBloc`.

Класс `LocaleState` представляет текущее состояние локализации приложения. Он содержит одно поле:

- `currentLocale` – объект типа `Locale`, который хранит текущую локаль приложения.

Класс `LocaleState` использует пакет `Equatable` для корректного сравнения состояний, а также `copy_with_extension`, который генерирует метод `copyWith`. Этот метод позволяет создавать новые экземпляры состояния с изменёнными значениями только тех полей, которые необходимы, не изменяя остальные, что упрощает управление состоянием в блоке.

Класс `LocaleEvent` является базовым для всех событий, связанных с локализацией. Производный класс `ChangeLocaleEvent` инициирует смену текущей локали на другую из поддерживаемых.

Класс `LocaleBloc` реализует бизнес-логику для управления локализацией в приложении. Он принимает начальную локаль через конструктор и устанавливает её в состоянии. Блок обрабатывает событие `ChangeLocaleEvent` с помощью метода `_onChangeLocale`. Этот метод ищет следующую локаль из списка поддерживаемых языков, отличную от текущей, и обновляет состояние. После этого с помощью метода `emit` создаётся новое состояние, которое включает в себя изменённую локаль.

Слой бизнес-логики для работы с реакциями (лайками и дизлайками) в приложении также реализован с использованием паттерна `BLoC`, что обеспечивает четкое разделение логики и представления, а также эффективное управление состоянием. В его состав входят три основных компонента: состояние, события и сам блок `ReactionBloc`.

Класс `ReactionState` представляет текущее состояние реакций и включает два поля: `likedIds` и `dislikedIds`, которые содержат списки идентификаторов объектов, получивших лайк или дизлайк соответственно. Оба списка могут быть равны `null`, но по умолчанию инициализируются пустыми списками. Класс использует пакет `Equatable` для корректного сравнения состояний и генерирует метод `copyWith`, который позволяет создавать новые экземпляры состояния с изменёнными значениями только нужных полей.

События, связанные с реакциями, включают:

- `LoadReactionsEvent` – для загрузки лайков и дизлайков из локального хранилища.
- `ChangeLikeEvent` – для изменения состояния лайка на объекте.
- `ChangeDislikeEvent` – для изменения состояния дизлайка на объекте.

Класс `ReactionBloc` управляет бизнес-логикой реакций. В конструкторе блока устанавливается начальное состояние с пустыми списками для лайков и дизлайков. Он обрабатывает события следующим образом:

- `_onLoadReactions` загружает лайки и дизлайки из `SharedPreferences` и обновляет состояние.
- `_onChangeLike` добавляет или удаляет лайк для указанного объекта, при этом удаляя дизлайк, если он был установлен. Изменения сохраняются в `SharedPreferences`.
- `_onChangeDislike` работает аналогично для дизлайков, удаляя лайк, если он был установлен.

Все изменения состояния выполняются с помощью метода `emit`, что приводит к обновлению UI. Важной особенностью реализации является использование `copyWith` для создания новых состояний с изменёнными полями, а также использование `Equatable` для корректного сравнения состояний и предотвращения лишних перерисовок UI. Данные реакций сохраняются в `SharedPreferences`, что позволяет сохранять лайки и дизлайки между сессиями приложения.

Слой представления

Модуль `svg_objects.dart` из пакета общих компонентов `common` предоставляет инструменты для работы с SVG-изображениями в приложении, включая их предварительную загрузку и интеграцию в пользовательский интерфейс. Он состоит из статического класса `SvgObjects`, отвечающего за инициализацию ресурсов, и двух виджетов (`SvgRu` и `SvgUk`), представляющих конкретные изображения.

Класс `SvgObjects` реализует метод `init`, который выполняет предварительную загрузку SVG-файлов для обеспечения их быстрой доступности во время работы приложения. Этот метод определяет список ресурсов, содержащихся в `resources.g.dart`, и загружает их с помощью `SvgAssetLoader`. Результаты загрузки сохраняются в кэше, что снижает задержки при последующем использовании изображений.

Виджеты `SvgRu` и `SvgUk` служат для отображения конкретных SVG-изображений, таких как флаги, через компонент `SvgPicture.asset` библиотеки `flutter_svg`. Они реализуют метод `build`, возвращающий готовый виджет для вставки в пользовательский интерфейс. Эти виджеты позволяют легко и удобно использовать заранее определенные SVG-ресурсы.

Модуль `dialogs`, предназначенный для отображения сообщений об ошибках, предоставляет простой и удобный механизм для уведомления пользователей о возникших проблемах через модальные диалоговые окна. Он состоит из двух компонентов: функции `showErrorDialog` и виджета `ErrorDialog`, которые вместе обеспечивают удобство вызова и единообразное визуальное оформление ошибок.

Функция `showErrorDialog` выступает интерфейсом для отображения окна ошибок. Она принимает текущий контекст приложения (`context`) и текст ошибки (`error`), который нужно показать пользователю. Функция использует метод `showDialog` для отображения модального окна с экземпляром виджета `ErrorDialog`. Это упрощает вызов окна из различных частей приложения, позволяя разработчикам сосредоточиться только на передаче контекста и текста ошибки.

Виджет `ErrorDialog` отвечает за визуальное оформление окна с ошибкой. Он строит диалог, состоящий из серого контейнера с закругленными углами, внутри которого располагаются значок ошибки, символизирующий проблему, и текст сообщения. Если текст ошибки не передан, выводится сообщение по умолчанию – "UNKNOWN". Для повышения удобства восприятия элементы снабжены отступами, а текст оформлен в соответствии с общей темой приложения.

Компонент `HomePage` является основным экраном приложения, на котором отображаются карточки с возможностью взаимодействия через лайки, дизлайки и переход на страницы с подробным описанием. В этом классе присутствует поле `title`, представляющее собой строку, которая передается в конструктор и отображается в `AppBar` как заголовок страницы. При инициализации компонента в методе `initState` генерируется случайный цвет для фона `AppBar` с помощью метода `_generateColor`, который случайным образом изменяет компоненты RGB. Также в этом методе инициализируются события для загрузки данных и реакций через соответствующие `BLoC`: `HomeLoadDataEvent` и `LoadReactionsEvent`.

Основной метод компонента – `build`, который строит структуру страницы, включая отображение `AppBar` с динамическим цветом, строку поиска и список карточек. Строка поиска, представленная компонентом `CupertinoSearchTextField`, осуществляет поиск карточек по имени персонажей. В зависимости от состояния данных, получаемых через `BLoC`, отображаются индикаторы загрузки, ошибки или сами карточки. При этом используется `BlocBuilder` для подписки на изменения состояний `HomeBloc` и `ReactionBloc`. Метод `_generateColor` генерирует случайный цвет для фона, используя генератор случайных чисел, чтобы создать значение `Color` с компонентами RGB в пределах от 150 до 255, что дает светлые цвета для фона. В методах `_onLike` и `_onDislike` происходит обработка нажатий на кнопки лайков и дизлайков соответственно. Эти методы вызывают соответствующие события в `ReactionBloc` – `ChangeLikeEvent` и `ChangeDislikeEvent`, что обновляет состояние реакций на карточки. После этого отображается `SnackBar`,

информирующий пользователя о том, что его реакция была изменена. Метод `_navToDetails` инициирует переход на страницу с подробным описанием карточки персонажа, передавая в конструктор `DetailsPage` данные о выбранной карточке.

Метод `_showSnackBar` отображает уведомление пользователю о том, что его реакция на карточку персонажа была изменена. Он формирует сообщение с помощью локализованного текста, который зависит от состояния лайков и дизлайков. Метод `_onNextPageListener` отслеживает прокрутку списка и подгружает новые данные при достижении конца списка. Когда пользователь достигает конца, и если пагинация еще не загружается, вызывается событие для загрузки следующей страницы данных с учетом текущего значения поиска.

Класс `_Body` включает поля, такие как `searchController`, контролирующий ввод текста в строке поиска, и `scrollController`, который отслеживает прокрутку списка. В методе `initState` происходит инициализация SVG-объектов с помощью `SvgObjects.init()` и отправка событий для загрузки данных и реакций. Метод `dispose` очищает ресурсы, связанные с контроллерами. В методе `build` создается структура страницы, в том числе строка поиска и кнопка для смены языка интерфейса. В зависимости от состояния данных отображаются индикаторы загрузки, ошибки или сами карточки. При нажатии на кнопку для смены языка вызывается событие `ChangeLocaleEvent` для изменения локализации. Метод `_onRefresh` используется для обновления данных при жесте обновления, что инициирует повторную загрузку данных с текущим значением поиска.

Класс `_Card` представляет собой виджет, отображающий одну карточку персонажа. Он включает в себя поля, такие как `id`, `title`, `text`, представляющие идентификатор карточки, его заголовок и текст, а также поле `character`, которое хранит данные о персонаже. Также есть поля `isLiked` и `isDisliked`, которые отображают, был ли поставлен лайк или дизлайк на карточку, и коллбэки `onLike` и `onDislike`, которые обрабатывают изменения состояний лайков и дизлайков. Метод `fromData` является фабричным методом, который создает объект `_Card` из данных `CardData`. Он инициализирует все поля карточки,

включая состояние лайков и дизлайков. Методы `_onLike` и `_onDislike` вызывают соответствующие коллбэки для изменения реакций на карточку. Метод `build` отвечает за построение визуального представления одной карточки, включая аватар персонажа, его имя, краткую информацию по персонажу и кнопки для лайков и дизлайков. Карточка отображается в контейнере с тенью и округлыми углами, а также с возможностью прокрутки, если карточек слишком много.

`DetailsPage` – это экран, отображающий подробности о конкретной карточке, открывающийся при нажатии на выбранную карточку. В нем используется объект `CardData`, переданный в конструктор класса, который содержит информацию о карточке, включая информацию о персонаже, его аватар и заголовок.

В этом классе единственное поле – `model`, которое представляет собой экземпляр класса `CardData`. Это поле передается через конструктор и используется для отображения информации на экране.

Метод `build` создает структуру страницы, включающую в себя `AppBar`, на котором отображается имя персонажа, а также тело страницы, состоящее из различных виджетов. В теле страницы используется `Padding`, чтобы создать отступы вокруг контента, и `Column` для размещения дочерних виджетов. Основные элементы интерфейса – это аватар персонажа, заголовок карточки и сам текст с описанием персонажа. Аватар отображается через `Image.network` (если URL доступен) или через пустое серое изображение с иконкой по умолчанию, если аватар не задан.

Верхняя часть экрана включает `Stack`, который размещает аватар персонажа в левом верхнем углу, с наложением на него имени персонажа, стилизованного с использованием `Container` с полупрозрачным черным фоном. В этой области отображается имя персонажа, сокращенное до 13 символов, если оно длиннее этого значения. Это сокращение реализовано в методе `_shortenNickname`, который принимает строку и возвращает обрезанную версию имени с добавлением многоточия в конце, если длина имени превышает указанный предел.

Заголовок и текст карточки отображаются в нижней части страницы с отступами между ними, и каждый из них стилизован с использованием стандартных размеров шрифта и жирного начертания для заголовка.

Руководство пользователя

Приложение является платформой для взаимодействия с коллекцией персонажей из вселенной «Гарри Поттера», предоставляя пользователям удобный доступ к карточкам, каждая из которых содержит информацию о персонаже, его имя, изображение и описание. Основной целью приложения является создание удобного интерфейса для пользователей, которые хотят изучить мир персонажей из этой популярной вселенной, а также выразить свое отношение к каждому персонажу через лайки и дизлайки.

Приложение ориентировано на поклонников серии книг и фильмов о Гарри Поттере, включая как долгосрочных фанатов, так и тех, кто только начинает знакомиться с этой вселенной. Целевая аудитория включает любителей фэнтези, а также людей, которые ценят глубокие персонажные исследования и хотят больше узнать о героях, злодеях и других значимых фигурах из мира «Гарри Поттера».

Функциональность приложения включает несколько важных возможностей. Во-первых, пользователи могут искать персонажей по ключевым словам, что позволяет быстро найти карточки, соответствующие их интересам. Во-вторых, каждая карточка персонажа содержит основные данные: имя, изображение, описание, что позволяет пользователю легко ознакомиться с персонажем. В-третьих, приложение предоставляет возможность ставить лайки или дизлайки персонажам, а также просматривать реакцию других пользователей на этих персонажей.

Дополнительно, приложение имеет страницу подробной информации о каждом персонаже, где представлены более углубленные данные: краткие биографические сведения и описание персонажа. Это позволяет пользователям получить более полное представление о каждом герое или злодее и углубиться в их роль в истории.

Для удобства пользователей, приложение поддерживает возможность переключения между языками, что позволяет использовать его как для

русскоязычных, так и для англоязычных пользователей, увеличивая доступность и удобство взаимодействия с контентом.

Основные варианты использования:

- **Просмотр карточек персонажей:** каждая карточка содержит информацию о персонаже, включая его имя, описание и изображение. Это позволяет пользователям быстро ознакомиться с внешностью персонажа и его ролью в мире «Гарри Поттера». В карточках отображаются как положительные, так и отрицательные персонажи, что дает возможность пользователю увидеть широкий спектр героев и злодеев.
- **Просмотр подробной информации о персонаже:** при нажатии на карточку персонажа пользователь попадает на страницу с детализированной информацией. Эта страница включает в себя более глубокое описание персонажа. Это дает возможность пользователю узнать больше о каждом герое или злодее, а также лучше понять их мотивацию и роль в мире Гарри Поттера.
- **Выражение мнения о персонажах через лайки и дизлайки:** пользователи могут ставить лайки или дизлайки персонажам, что дает возможность активно участвовать в оценке персонажей. Это позволяет фанатам выразить свои предпочтения. Лайк и дизлайк могут быть использованы для того, чтобы показать отношение к персонажу – нравится ли он пользователю или наоборот вызывает отрицательные эмоции.
- **Поиск персонажей по имени или ключевым словам:** пользователь может воспользоваться встроенной строкой поиска, чтобы быстро найти карточку нужного персонажа. Это позволяет сэкономить время при поиске определенных героев или злодеев, а также дает возможность найти интересующую информацию по запросу. По мере ввода текста приложение автоматически обновляет список персонажей, соответствующих введенному поисковому запросу.

- **Переключение между языками:** для удобства пользователей приложение поддерживает два языка: русский и английский. Пользователь может в любой момент сменить язык интерфейса, что помогает адаптировать приложение для пользователей, говорящих на этих языках. Это обеспечивает доступность приложения для широкой аудитории.
- **Обновление контента и реакции на персонажей:** приложение обновляется в режиме реального времени. При прокрутке списка карточек на экран подгружаются новые данные о персонажах, что позволяет пользователю получить всегда актуальную информацию.

После запуска приложения пользователь увидит главный экран, на котором будет отображаться список карточек персонажей вселенной «Гарри Поттера». Важно отметить, что для корректной работы приложения необходимо наличие стабильного подключения к интернету, так как все данные, включая изображения и описания персонажей, загружаются из удаленного источника. При отсутствии подключения к интернету приложение не сможет загрузить данные и предложит пользователю подключиться к сети.

Сразу после запуска приложения начинается процесс загрузки информации о персонажах. В случае отсутствия интернета приложение отобразит уведомление, предлагающее подключиться к сети. После успешной загрузки данных на экране появится список карточек.

Главный экран приложения представляет собой центральное место взаимодействия пользователя с информацией о персонажах из вселенной «Гарри Поттера». Его интерфейс организован так, чтобы обеспечить удобный доступ ко всем функциям и упростить навигацию.

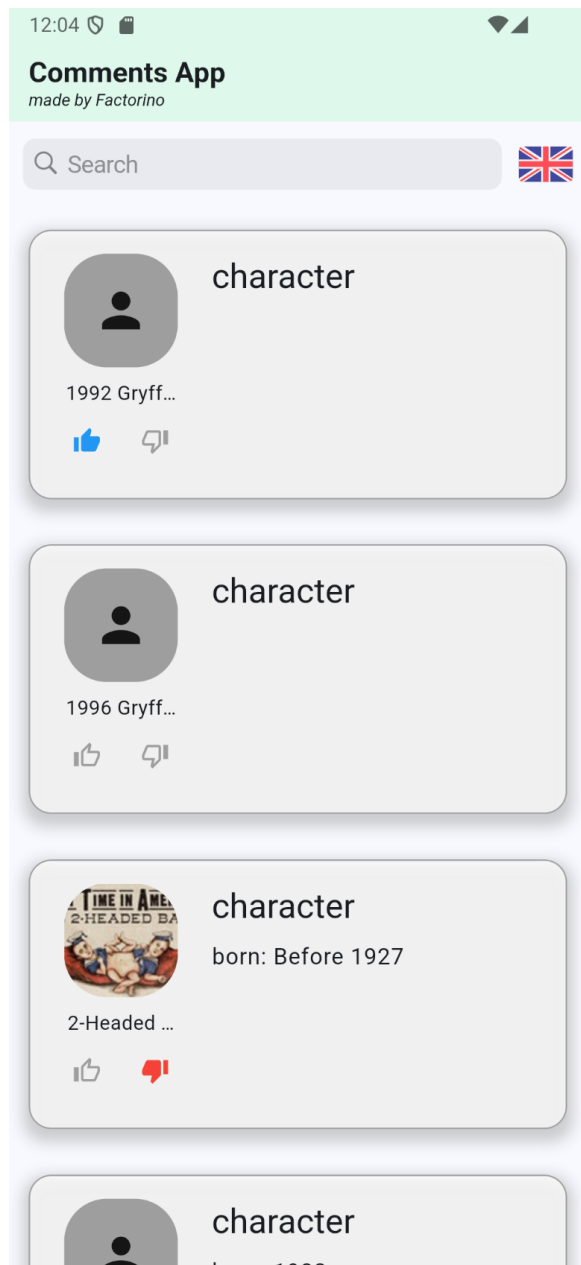


Рисунок 7 Главный экран

Основной элемент экрана – вертикальный список карточек персонажей, каждая из которых включает изображение, имя и краткое описание персонажа. Если изображение недоступно, вместо него отображается стандартная заглушка с иконкой, что позволяет поддерживать единообразие интерфейса. Пользователь может свободно прокручивать список, чтобы просматривать доступные карточки.

Карточка персонажа в приложении представляет собой структурированный элемент интерфейса, который содержит изображение, текстовую информацию и элементы взаимодействия. Она визуально разделена на две части: левую и правую.

В левой части карточки расположено изображение персонажа, оформленное в круглом обрамлении с радиусом закругления. Размер изображения составляет 80x80 пикселей, и оно загружается из сети, если указана ссылка. В случае отсутствия изображения отображается стандартная заглушка в виде серого фона с иконкой человека.

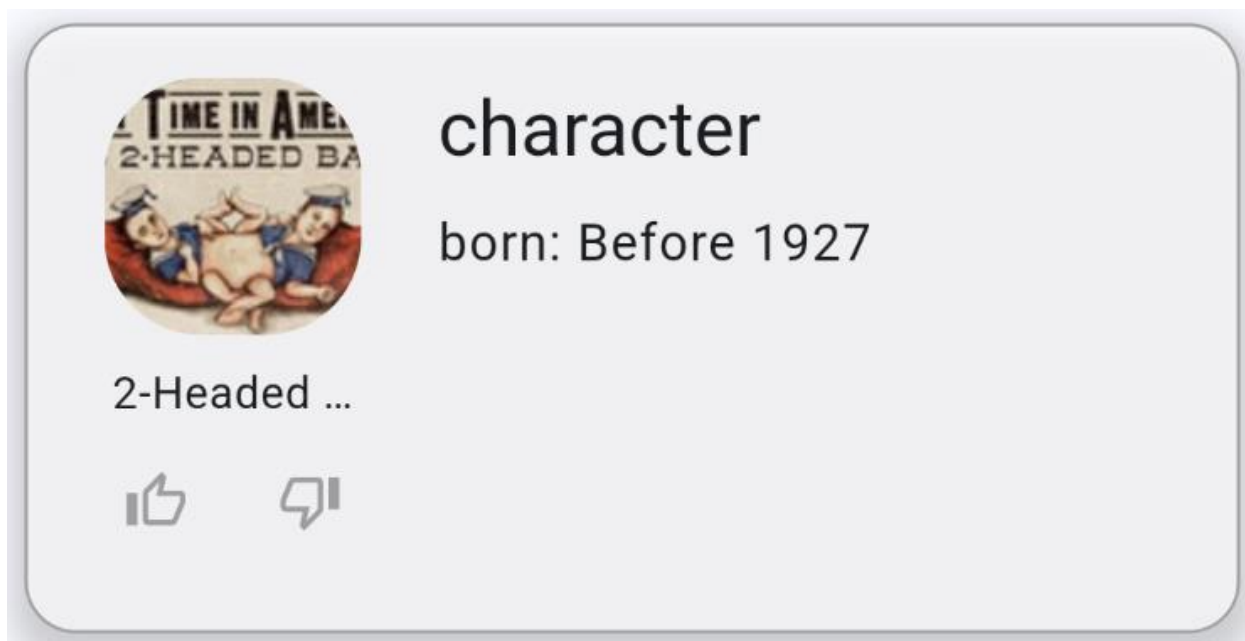


Рисунок 8. Карточка персонажа (с изображением)

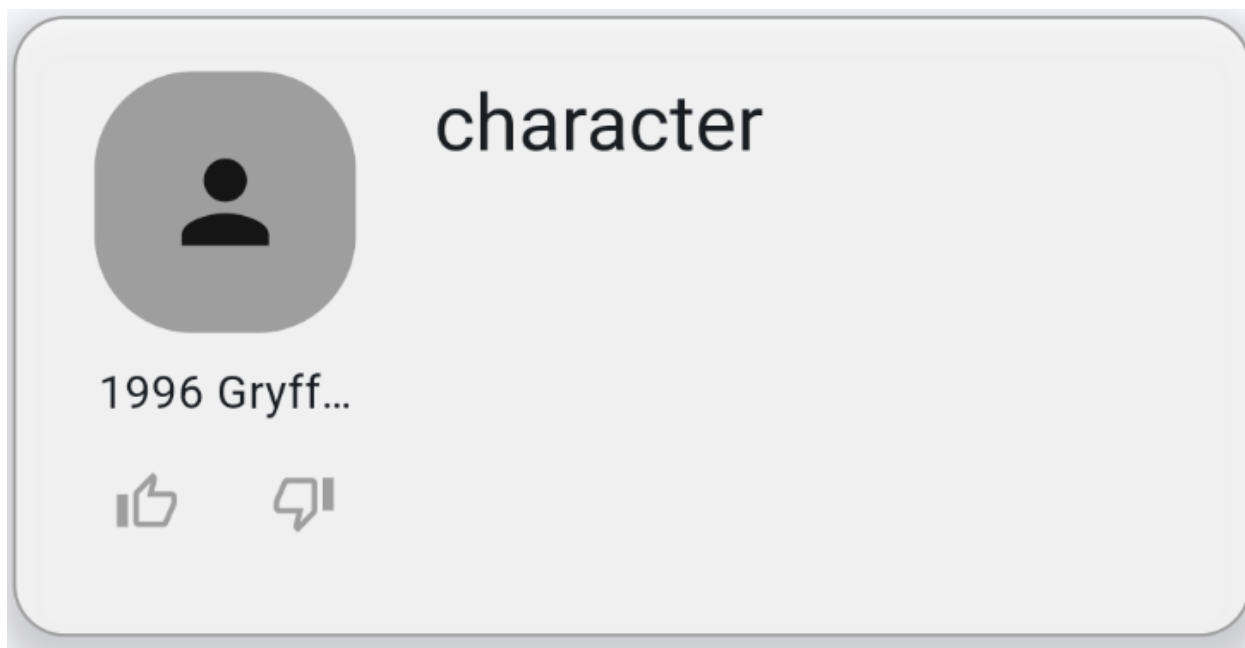


Рисунок 9. Карточка персонажа (с заглушкой)

Под изображением находится текстовое поле с именем персонажа, выровненное по центру. Имя ограничено одной строкой с добавлением многоточия, если оно не помещается. Ширина блока имени фиксирована (80 пикселей) для обеспечения симметрии. Под текстовым блоком располагаются

кнопки для оценки персонажа: «лайк» и «дизлайк». Каждая кнопка представлена соответствующей иконкой, состояние которых меняется в зависимости от действия пользователя. Лайк отображается синим цветом при активации, а дизлайк – красным. Неактивные кнопки подсвечиваются серым. Эти кнопки позволяют пользователю выразить своё отношение к персонажу.

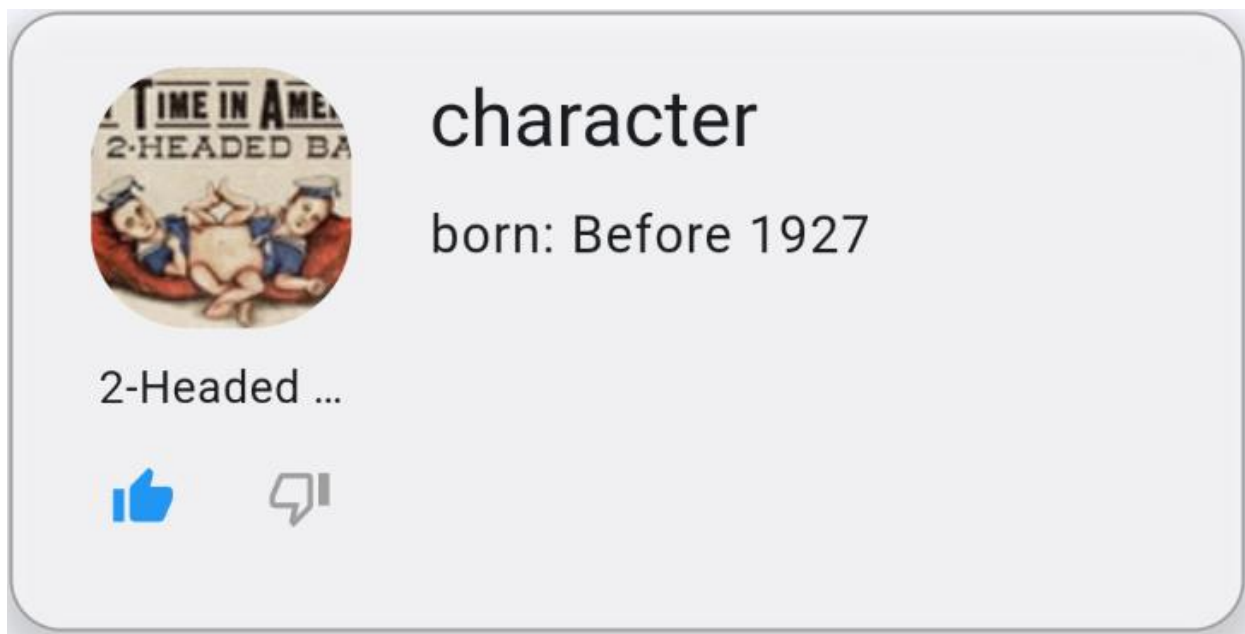


Рисунок 10. Отображение лайка на карточке

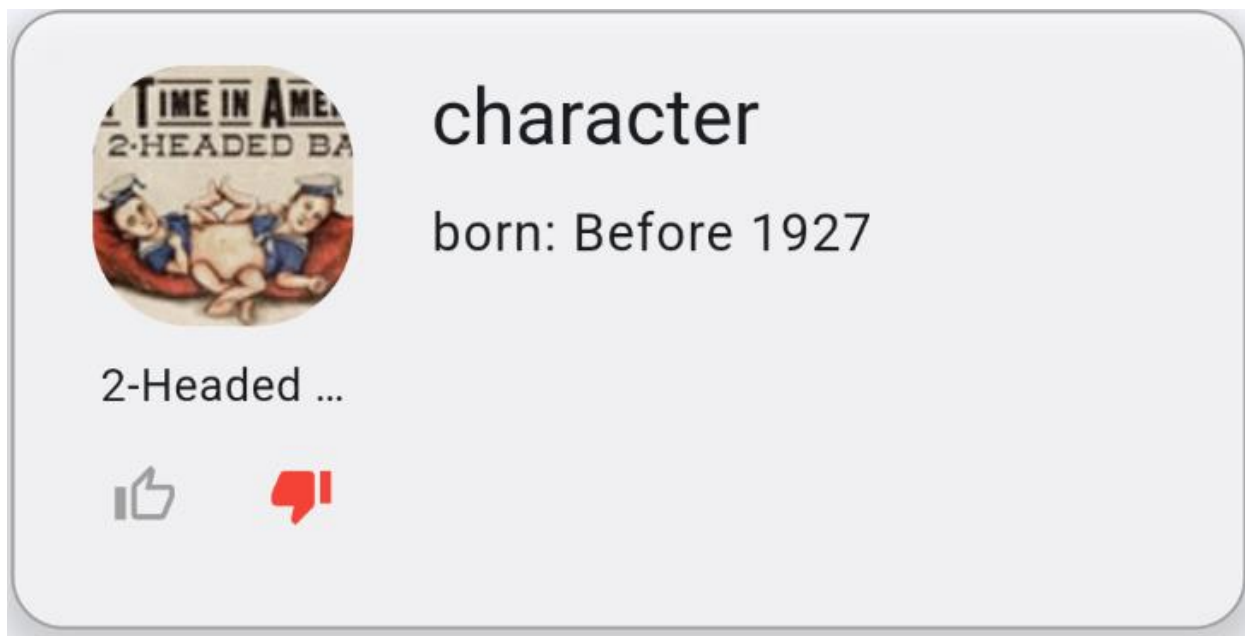


Рисунок 11. Отображение дизлайка на карточке

Правая часть карточки предназначена для текстовой информации. В её верхней области располагается заголовок, который выделен крупным шрифтом и представляет собой тип информации «character». Заголовок предоставляет общую характеристику персонажа, позволяя пользователю

быстро понять, о ком идёт речь. Ниже заголовка размещён текстовый блок с описанием, где содержится краткая биографическая информация о персонаже: дата рождения и дата смерти. Текст отображается в несколько строк, предоставляя достаточный объём информации для общего понимания.

Карточка оформлена с использованием светлого фона, скруглённых углов и тонкой серой рамки. Для улучшения визуального восприятия добавлена лёгкая тень, создающая эффект приподнятости и отделяющая карточку от фона. Отступы между элементами карточки соблюдены для обеспечения удобства чтения: изображение, текст и кнопки взаимодействия разделены фиксированным пространством. Вся структура карточки организована так, чтобы пользователь мог легко воспринимать ключевую информацию, оценивать персонажей и взаимодействовать с ними.

Если пользователь хочет узнать больше о конкретном персонаже, он может нажать на соответствующую карточку, чтобы перейти на страницу с детальной информацией. На этой странице отображается расширенное описание персонажа.

Экран детальной информации персонажа организован так, чтобы пользователи могли удобно и быстро получить полные сведения о выбранном герое. Верхнюю часть экрана занимает панель навигации, где отображается имя персонажа, представленное в виде заголовка, который подстраивается под язык интерфейса. Это позволяет сразу понять, о каком персонаже идет речь.

Основное содержимое экрана разделено на три ключевых блока, размещенных вертикально. В верхней части находится изображение персонажа, оформленное как квадрат с закругленными углами. Если доступно изображение, оно загружается с указанного URL и отображается в фиксированном размере 150x150 пикселей. Если изображение недоступно, на его месте отображается серый контейнер с иконкой человека размером 50 пикселей. Поверх изображения, в его левом нижнем углу, добавлен полупрозрачный чёрный прямоугольник с именем персонажа, которое обрезается, если превышает 13 символов, и заканчивается многоточием. Текст

выполнен белым цветом, жирным шрифтом, что делает его заметным, но ненавязчивым.

Под изображением персонажа расположен заголовок, отражающий название карточки. Он выделен крупным жирным шрифтом, что привлекает внимание пользователя. Заголовок находится на комфортном расстоянии от изображения благодаря вертикальному отступу, что обеспечивает логическую разделенность информации.

Ниже заголовка находится текстовое описание персонажа, представляющее собой основную информацию о нем. Текст выполнен стандартным шрифтом, удобным для чтения. Между заголовком и описанием предусмотрен отступ, чтобы визуально отделить эти блоки.

Все элементы на экране выровнены по левой стороне для создания ощущения логической последовательности. Пространство экрана окружено отступами, что помогает визуально отделить содержимое от краев устройства, сохраняя комфорт восприятия. Дизайн экрана минималистичен и интуитивно понятен, позволяя пользователю сосредоточиться на представленной информации без лишних отвлекающих элементов.

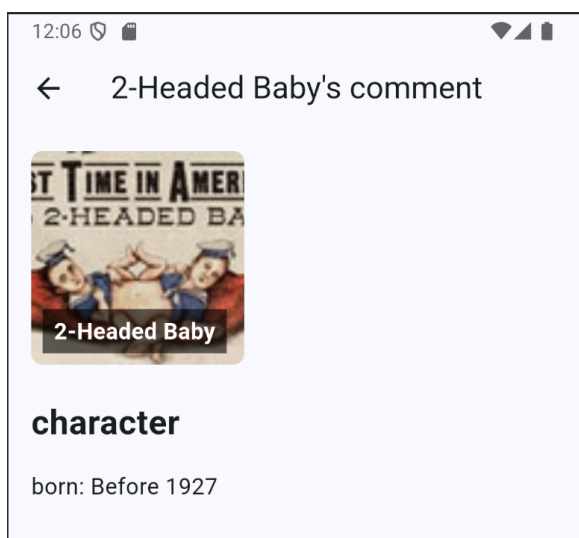


Рисунок 12. Экран детальной информации (с изображением)

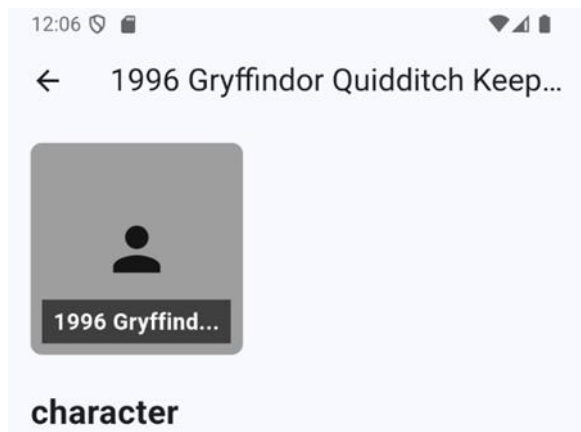


Рисунок 13. Детальный экран (с заглушкой)

В верхней части экрана расположена строка поиска, предназначенная для быстрого нахождения интересующего персонажа. Пользователь может вводить имя персонажа или ключевые слова, связанные с ним, а приложение будет автоматически фильтровать отображаемые карточки, показывая только те, которые соответствуют запросу. Это особенно удобно, если список персонажей велик или пользователь ищет конкретного героя или злодея.

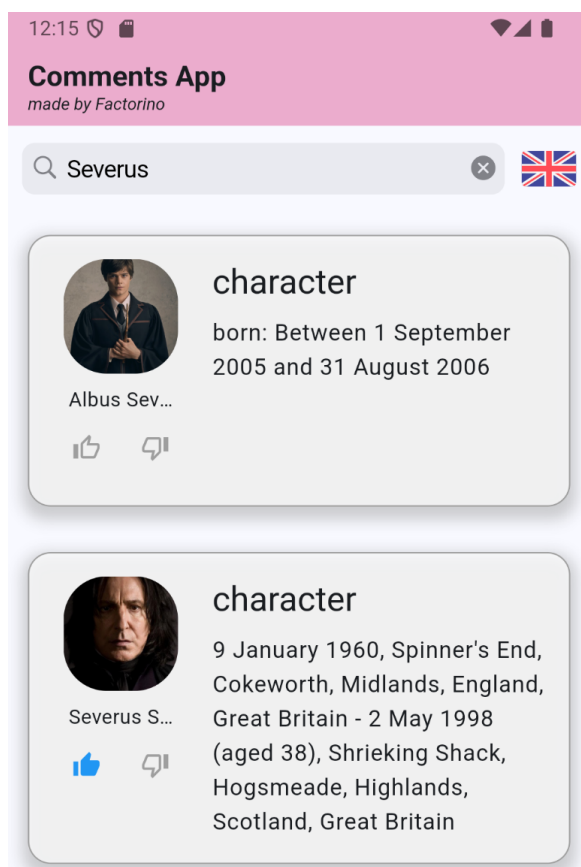


Рисунок 14. Поиск карточек по имени

Рядом со строкой поиска находится кнопка переключения языка, оформленная в виде иконки с изображением флага. Нажатие на эту кнопку позволяет сменить язык интерфейса между русским и английским. Это делает

приложение удобным для пользователей, говорящих на этих языках, и расширяет его доступность для международной аудитории.

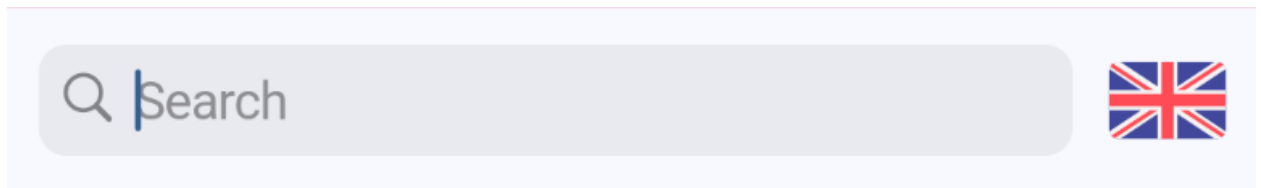


Рисунок 15. Английский язык

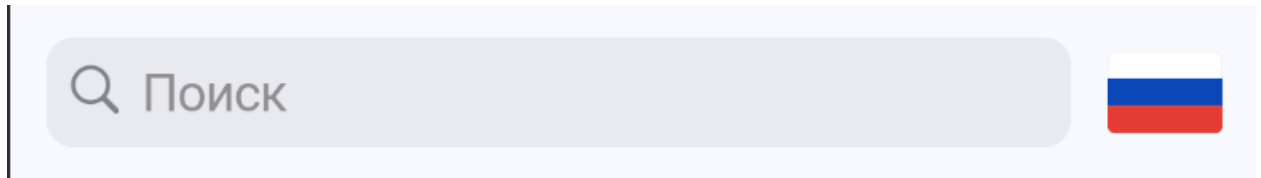


Рисунок 16. Русский язык

Контент на главном экране обновляется в реальном времени. При прокрутке вниз автоматически подгружаются новые карточки, а при выполнении жеста «потянуть вниз» пользователь может вручную обновить данные, чтобы получить актуальную информацию. Реакции пользователей, такие как лайки и дизлайки, мгновенно сохраняются в системе, что позволяет сохранять актуальность данных без необходимости перезагрузки приложения.

Для того чтобы продемонстрировать использование приложения от момента его запуска до достижения конечного результата, рассмотрим сценарий взаимодействия с приложением на примере поиска и оценки персонажа из вселенной «Гарри Поттера».

- 1. Запуск приложения:** пользователь открывает приложение. На экране появляется главный экран, содержащий список карточек персонажей. В верхней части экрана расположен текстовый поиск и иконка переключения языка. Список карточек подгружается автоматически при наличии интернет-соединения, отображая основную информацию о каждом персонаже, такую как имя, изображение и краткое описание.
- 2. Поиск персонажа:** допустим, пользователь ищет карточку с персонажем Гарри Поттером. Он вводит в строку поиска слово «Harry». Приложение мгновенно фильтрует список и оставляет только те карточки, которые соответствуют введенному запросу. Пользователь видит карточку Гарри Поттера среди результатов.

- 3. Открытие подробной информации:** пользователь нажимает на карточку Гарри Поттера. Приложение переходит на экран детальной информации. Здесь пользователь видит:
- Изображение персонажа (или заглушку, если изображение отсутствует);
 - Имя персонажа;
 - Заголовок карточки «character», сообщающая пользователю о том, что он просматривает информацию именно о персонаже;
 - Краткое описание персонажа, содержащее его дату рождения и дату смерти.
- 4. Оценка персонажа:** возвратившись на главный экран, пользователь решает выразить свое мнение о персонаже. Он ставит лайк на карточке Гарри Поттера, нажав на соответствующую кнопку с иконкой поднятого вверх большого пальца. Иконка лайка становится активной (окрашивается в синий цвет), что подтверждает успешное действие.
- 5. Изучение других персонажей:** после оценки персонажа пользователь может пролистать список вниз, чтобы увидеть другие карточки. Если список достигает конца текущей страницы, приложение автоматически подгружает новые данные.
- 6. Переключение языка:** предположим, пользователь хочет переключить язык интерфейса на английский. Он нажимает на иконку флага в верхней правой части экрана. Интерфейс мгновенно переключается на английский язык, включая все подписи интерфейса.
- 7. Завершение работы:** после завершения взаимодействия пользователь закрывает приложение. Все действия, такие как лайки и дизлайки, сохраняются, и при следующем запуске приложения пользователь видит обновленное состояние.

Заключение

В рамках данной курсовой работы было разработано мобильное приложение, посвящённое персонажам вселенной «Гарри Поттера». Целью проекта было создание интерактивного инструмента, который позволит фанатам франшизы глубже познакомиться с миром волшебства, а также предоставит удобный и персонализированный опыт работы с информацией о персонажах. Приложение включает в себя несколько ключевых функций:

- **Просмотр карточек персонажей:** пользователь может просматривать список карточек, каждая из которых содержит краткую информацию о персонаже: его имя, изображение и краткое описание. Это позволяет быстро ознакомиться с основными героями и злодеями вселенной «Гарри Поттера». Карточки включают как положительных персонажей, так и антагонистов, что дает полное представление о разнообразии персонажей.
- **Просмотр детальной информации о персонаже:** при нажатии на карточку персонажа открывается страница с более детализированным описанием. Здесь пользователь может ознакомиться с биографией персонажа. Это помогает глубже понять роль персонажа в истории.
- **Выражение мнения о персонаже (лайки и дизлайки):** пользователи могут выражать свое мнение о персонажах, ставя лайки или дизлайки на карточках. Это позволяет активно участвовать в оценке персонажей и демонстрировать свои предпочтения. Все поставленные лайки и дизлайки сохраняются в локальном хранилище (SharedPreferences), что позволяет отображать предпочтения пользователя при каждом новом запуске приложения.
- **Поиск персонажей по имени или ключевым словам:** встроенная строка поиска позволяет пользователю быстро найти

карточки персонажей по имени или другим ключевым словам. По мере ввода текста результаты поиска обновляются в реальном времени, что значительно ускоряет процесс поиска нужного персонажа среди множества других.

- **Переключение между языками:** для удобства пользователей приложение поддерживает два языка интерфейса: русский и английский. Пользователь может выбрать предпочтительный язык в настройках, что делает приложение доступным для более широкой аудитории и помогает адаптировать его для разных регионов.
- **Обновление контента и пагинация:** при прокрутке списка карточек автоматически подгружаются новые данные о персонажах, что обеспечивает плавное и динамичное взаимодействие с приложением. Это позволяет пользователю получать актуальную информацию, не перегружая интерфейс и обеспечивая удобство при работе с большими объемами данных.

Разработка мобильного приложения включала несколько ключевых этапов, которые позволили создать полноценный продукт, отвечающий всем требованиям и ожиданиям пользователей. В процессе разработки было осуществлено тщательное планирование и проектирование, чтобы обеспечить качественную работу приложения на всех этапах.

Основным первым этапом стала анализа предметной области, в рамках которого было изучено содержание франшизы «Гарри Поттер», собраны данные о персонажах и их характеристиках, которые должны были быть представлены в приложении. Были выделены ключевые сущности, такие как персонажи, карточки с краткой информацией, детализированные профили персонажей и система оценок (лайки и дизлайки), а также определены основные функциональные возможности, включая поиск, локализацию интерфейса и пагинацию.

После анализа предметной области был выполнен этап проектирования приложения. На этом этапе была разработана архитектура приложения,

выбраны технологии и инструменты для реализации. Была принята концепция разделения бизнес-логики и представления с использованием паттерна BLoC. Для обеспечения комфортного пользовательского опыта был спроектирован интерфейс с учётом всех функциональных требований, включая главный экран с карточками персонажей, детализированную информацию, поиск, настройки локализации и систему реакций. Также было принято решение использовать Flutter для кросс-платформенной разработки и SharedPreferences для сохранения данных о реакциях пользователей.

Следующим этапом стала реализация приложения, включающая написание кода и интеграцию всех компонентов. В ходе реализации были созданы экраны для отображения карточек персонажей, функции детализированного просмотра, а также элементы управления для поиска персонажей и работы с реакциями (лайки и дизлайки). Важной частью реализации стало подключение удалённого API для загрузки данных о персонажах и поддержка пагинации, что обеспечило стабильную работу приложения при увеличении объёмов данных. Также была добавлена возможность изменения локализации, поддерживающая два языка – русский и английский.

Завершающим этапом стала оптимизация и рефакторинг. Этот процесс включал в себя улучшение читаемости и производительности кода, оптимизацию работы с данными и повышение стабильности приложения.

На основе реализованного приложения можно строить дальнейшие улучшения и расширения, добавляя новые функции, такие как интеграция с новыми медиаформатами (например, видео или аудио контентом), создание персонализированных рекомендаций для пользователей, а также более глубокую интеграцию с внешними источниками данных. Проект является хорошей основой для дальнейшего развития и углубления функционала, обеспечивая при этом удобство и стабильность работы на всех этапах взаимодействия с пользователем.

Список литературы

1. Bloc Library. Flutter Bloc Documentation [Электронный ресурс]. Режим доступа: <https://bloclibrary.dev/#/gettingstarted> (дата обращения: 20.12.2024).
2. Dart Language Specification [Электронный ресурс]. Режим доступа: <https://dart.dev/guides/language/spec> (дата обращения: 20.12.2024).
3. Flutter. Official documentation. Google Developers [Электронный ресурс]. Режим доступа: <https://flutter.dev/docs> (дата обращения: 20.12.2024).
4. Material Design Guidelines. Google Developers [Электронный ресурс]. Режим доступа: <https://m3.material.io/> (дата обращения: 20.12.2024).
5. Введение в i18n и l10n в приложениях Flutter [Электронный ресурс]. Режим доступа: <https://pub.dev/packages/intl> (дата обращения: 20.12.2024).
6. Интеграция REST API в Flutter-приложениях [Электронный ресурс]. Режим доступа: <https://flutter.dev/docs/cookbook/networking/fetch-data> (дата обращения: 20.12.2024).
7. Михаил А. Управление состоянием приложения в Flutter // Хабр [Электронный ресурс]. Режим доступа: <https://habr.com/ru/articles/435780/> (дата обращения: 20.12.2024).
8. Принципы асинхронного программирования в Dart [Электронный ресурс]. Режим доступа: <https://dart.dev/codelabs/async-await> (дата обращения: 20.12.2024).
9. Репозиторий с исходным кодом. [Электронный ресурс]. Режим доступа: https://git.is.ulstu.ru/Factorino73/PIbd-31_MasenkinMS_PMU/src/branch/Coursework (дата обращения: 20.12.2024).

10. Ресурсы для работы с SVG в Flutter [Электронный ресурс]. Режим доступа: https://pub.dev/packages/flutter_svg (дата обращения: 20.12.2024).
11. Руководство по разработке и проектированию на Flutter [Электронный ресурс]. Режим доступа: <https://flutterawesome.com/> (дата обращения: 20.12.2024).

Исходный код

```
import
'package:flutter/widgets.dart';
import
'package:pmu_labworks/components/lo
cale/l10n/app_locale.dart';

extension LocalContextX on
BuildContext {
  AppLocale get locale =>
AppLocale.of(this)!;
}
import 'dart:async';

import
'package:flutter/foundation.dart';
import
'package:flutter/widgets.dart';
import
'package:flutter_localizations/flut
ter_localizations.dart';
import 'package:intl/intl.dart' as
intl;

import 'app_locale_en.dart';
import 'app_locale_ru.dart';

// ignore_for_file: type=lint

/// Callers can lookup localized
strings with an instance of
AppLocale
/// returned by
`AppLocale.of(context)`.
///
/// Applications need to include
`AppLocale.delegate()` in their
app's
/// `localizationDelegates` list,
and the locales they support in the
app's
/// `supportedLocales` list. For
example:
///
/// ```dart
/// import 'l10n/app_locale.dart';
///
/// return MaterialApp(
///   localizationsDelegates:
AppLocale.localizationsDelegates,
///   supportedLocales:
AppLocale.supportedLocales,
///   home: MyApplicationHome(),
/// );
/// ```
/// ## Update pubspec.yaml
///
/// Please make sure to update your
pubspec.yaml to include the
following
/// packages:
///
/// ```yaml
```

```
/// dependencies:
///   # Internationalization
support.
///   flutter_localizations:
///     sdk: flutter
///   intl: any # Use the pinned
version from flutter_localizations
///
///   # Rest of dependencies
///   ```
///
/// ## iOS Applications
///
/// iOS applications define key
application metadata, including
supported
/// locales, in an Info.plist file
that is built into the application
bundle.
/// To configure the locales
supported by your app, you'll need
to edit this
/// file.
///
/// First, open your project's
ios/Runner.xcworkspace Xcode
workspace file.
/// Then, in the Project Navigator,
open the Info.plist file under the
Runner
/// project's Runner folder.
///
/// Next, select the Information
Property List item, select Add Item
from the
/// Editor menu, then select
Localizations from the pop-up menu.
///
/// Select and expand the newly-
created Localizations item then,
for each
/// locale your application
supports, add a new item and select
the locale
/// you wish to add from the pop-up
menu in the Value field. This list
should
/// be consistent with the
languages listed in the
AppLocale.supportedLocales
/// property.
abstract class AppLocale {
  AppLocale(String locale) :
localeName =
intl.Intl.canonicalizedLocale(local
e.toString());

  final String localeName;

  static AppLocale? of(BuildContext
context) {
    return
Localizations.of<AppLocale>(context
```

```

, AppLocale);
}

static const
LocalizationsDelegate<AppLocale>
delegate = _AppLocaleDelegate();

/// A list of this localizations
delegate along with the default
localizations
/// delegates.
///
/// Returns a list of
localizations delegates containing
this delegate along with
///
GlobalMaterialLocalizations.delegat
e,
GlobalCupertinoLocalizations.delega
te,
/// and
GlobalWidgetsLocalizations.delegate
'.
///
/// Additional delegates can be
added by appending to this list in
/// MaterialApp. This list does
not have to be used at all if a
custom list
/// of delegates is preferred or
required.
static const
List<LocalizationsDelegate<dynamic>
> localizationsDelegates =
<LocalizationsDelegate<dynamic>>[
    delegate,

GlobalMaterialLocalizations.delegat
e,

GlobalCupertinoLocalizations.delega
te,

GlobalWidgetsLocalizations.delegate
',
];

/// A list of this localizations
delegate's supported locales.
static const List<Locale>
supportedLocales = <Locale>[
    Locale('en'),
    Locale('ru')
];

/// No description provided for
@title.
///
/// In ru, this message
translates to:
/// **'Комментарий'**
String get title;

/// No description provided for
@credits.
///

```

```

/// In ru, this message
translates to:
/// **'сделано'**
String get credits;

/// No description provided for
@search.
///
/// In ru, this message
translates to:
/// **'Поиск'**
String get search;

/// No description provided for
@comment.
///
/// In ru, this message
translates to:
/// **'Комментарий {nickname}'**
String comment(Object nickname);

/// No description provided for
@liked.
///
/// In ru, this message
translates to:
/// **'лайкнут'**
String get liked;

/// No description provided for
@disliked.
///
/// In ru, this message
translates to:
/// **'дизлайкнут'**
String get disliked;

/// No description provided for
@neutralized.
///
/// In ru, this message
translates to:
/// **'без реакции'**
String get neutralized;

/// No description provided for
@born.
///
/// In ru, this message
translates to:
/// **'Родился'**
String get born;

/// No description provided for
@died.
///
/// In ru, this message
translates to:
/// **'Умер'**
String get died;

/// No description provided for
@arbEnding.
///
/// In ru, this message
translates to:

```

```

    /// **'Чтобы не забыть про
    отсутствие запятой :)'**
    String get arbEnding;
}

class _AppLocaleDelegate extends
LocalizationsDelegate<AppLocale> {
  const _AppLocaleDelegate();

  @override
  Future<AppLocale> load(Locale
locale) {
    return
SynchronousFuture<AppLocale>(lookup
AppLocale(locale));
  }

  @override
  bool isSupported(Locale locale)
=> <String>['en',
'ru'].contains(locale.languageCode)
;

  @override
  bool
shouldReload(_AppLocaleDelegate
old) => false;
}

AppLocale lookupAppLocale(Locale
locale) {

  // Lookup logic when only
  language code is specified.
  switch (locale.languageCode) {
    case 'en': return
AppLocaleEn();
    case 'ru': return
AppLocaleRu();
  }

  throw FlutterError(
    'AppLocale.delegate failed to
load unsupported locale "$locale".
This is likely '
    'an issue with the
localizations generation tool.
Please file an issue '
    'on GitHub with a reproducible
sample app and the gen-l10n
configuration '
    'that was used.'
  );
}
import 'app_locale.dart';

// ignore_for_file: type=lint

/// The translations for English
('en').
class AppLocaleEn extends AppLocale
{
  AppLocaleEn([String locale =
'en']) : super(locale);

```

```

  @override
  String get title => 'Comments
App';

  @override
  String get credits => 'made by';

  @override
  String get search => 'Search';

  @override
  String comment(Object nickname) {
    return '$nickname\'s comment';
  }

  @override
  String get liked => 'liked';

  @override
  String get disliked =>
'disliked';

  @override
  String get neutralized =>
'neutralized';

  @override
  String get born => 'Born';

  @override
  String get died => 'Died';

  @override
  String get arbEnding => 'Чтобы не
забыть про отсутствие запятой :)';
}
import 'app_locale.dart';

// ignore_for_file: type=lint

/// The translations for Russian
('ru').
class AppLocaleRu extends AppLocale
{
  AppLocaleRu([String locale =
'ru']) : super(locale);

  @override
  String get title =>
'Комментарии';

  @override
  String get credits => 'сделано';

  @override
  String get search => 'Поиск';

  @override
  String comment(Object nickname) {
    return 'Комментарий $nickname';
  }

  @override
  String get liked => 'лайкнут';

  @override

```

```

String get disliked =>
'дизлайкнут';

@override
String get neutralized => 'без
реакции';

@override
String get born => 'Родился';

@override
String get died => 'Умер';

@override
String get arbEnding => 'Чтобы не
забыть про отсутствие запятой :)';
}
import 'dart:async';
import 'dart:ui';

class Debounce {
  factory Debounce() => _instance;

  Debounce._();

  static final Debounce _instance =
Debounce._();

  static Timer? _timer;

  static void run(
    VoidCallback action, {
    Duration delay = const
Duration(milliseconds: 500),
  }) {
    _timer?.cancel();
    _timer = Timer(delay, action);
  }
}

/// Generate by
[asset_generator] (https://github.com/fluttercandies/flutter\_asset\_generator) library.
/// PLEASE DO NOT EDIT MANUALLY.
// ignore_for_file:
constant_identifier_names
class R {
  const R._();

  ///
  ![preview] (file://C:\Users\masen\StudioProjects\pmu_labworks\assets\svg\ru.svg)
  static const String
ASSETS_SVG_RU_SVG =
'assets/svg/ru.svg';

  ///
  ![preview] (file://C:\Users\masen\StudioProjects\pmu_labworks\assets\svg\uk.svg)
  static const String
ASSETS_SVG_UK_SVG =
'assets/svg/uk.svg';
}

```

```

import 'character.dart';

class CardData {
  final String? id;
  final String title;
  final String text;
  final CharacterData character;

  CardData({
    this.id,
    required this.title,
    required this.text,
    required this.character,
  });
}

class CharacterData {
  final String name;
  final String? avatarUrl;

  CharacterData({
    required this.name,
    this.avatarUrl,
  });
}

import
'package:pmu_labworks/domain/models
/card.dart';

class HomeData {
  final List<CardData>? data;
  final int? nextPage;

  HomeData({this.data,
this.nextPage});
}

import
'package:json_annotation/json_annot
ation.dart';

part 'characters_dto.g.dart';

@JsonSerializable(createToJson:
false)
class CharactersDto {
  final List<CharacterDataDto>?
data;
  final MetaDto? meta;

  const CharactersDto({
    this.data,
    this.meta,
  });

  factory
CharactersDto.fromJson(Map<String,
dynamic> json) =>
_$_CharactersDtoFromJson(json);
}

@JsonSerializable(createToJson:
false)
class CharacterDataDto {
  final String? id;
  final String? type;
  final CharacterAttributesDataDto?
attributes;
}

```

```

    const CharacterDataDto({this.id,
this.type, this.attributes});

    factory
CharacterDataDto.fromJson(Map<String,
dynamic> json) =>
_$CharacterDataDtoFromJson(json);
}

@JsonSerializable(createToJson:
false)
class CharacterAttributesDataDto {
    final String? name;
    final String? born;
    final String? died;
    final String? image;

    const
CharacterAttributesDataDto({this.name,
this.born, this.died,
this.image});

    factory
CharacterAttributesDataDto.fromJson(
Map<String, dynamic> json) =>
_$CharacterAttributesDataDtoFromJson(
json);
}

@JsonSerializable(createToJson:
false)
class MetaDto {
    final PaginationDto? pagination;

    const MetaDto({this.pagination});

    factory
MetaDto.fromJson(Map<String,
dynamic> json) =>
_$MetaDtoFromJson(json);
}

@JsonSerializable(createToJson:
false)
class PaginationDto {
    final int? current;
    final int? next;
    final int? last;

    const
PaginationDto({this.current,
this.next, this.last});

    factory
PaginationDto.fromJson(Map<String,
dynamic> json) =>
_$PaginationDtoFromJson(json);
}
// GENERATED CODE - DO NOT MODIFY
BY HAND
part of 'characters_dto.dart';
//

```

```

*****
*****
****
// JsonSerializableGenerator
//
*****
*****
****

CharactersDto
_$CharactersDtoFromJson(Map<String,
dynamic> json) => CharactersDto(
    data: (json['data'] as
List<dynamic>?)
        ?.map((e) =>
CharacterDataDto.fromJson(e as
Map<String, dynamic>))
        .toList(),
    meta: json['meta'] == null ?
null :
MetaDto.fromJson(json['meta'] as
Map<String, dynamic>),
);

CharacterDataDto
_$CharacterDataDtoFromJson(Map<String,
dynamic> json) =>
CharacterDataDto(
    id: json['id'] as String?,
    type: json['type'] as
String?,
    attributes:
json['attributes'] == null
        ? null
        :
CharacterAttributesDataDto.fromJson(
json['attributes'] as Map<String,
dynamic>),
);

CharacterAttributesDataDto
_$CharacterAttributesDataDtoFromJson(
Map<String, dynamic> json) =>
CharacterAttributesDataDto(
    name: json['name'] as
String?,
    born: json['born'] as
String?,
    died: json['died'] as
String?,
    image: json['image'] as
String?,
);

MetaDto
_$MetaDtoFromJson(Map<String,
dynamic> json) => MetaDto(
    pagination:
json['pagination'] == null
        ? null
        :
PaginationDto.fromJson(json['pagina
tion'] as Map<String, dynamic>),
);

PaginationDto

```

```

_$PaginationDtoFromJson(Map<String,
dynamic> json) => PaginationDto(
  current: (json['current'] as
num?)?.toInt(),
  next: (json['next'] as
num?)?.toInt(),
  last: (json['last'] as
num?)?.toInt(),
);
import
'package:pmu_labworks/data/dtos/cha
racters_dto.dart';
import
'package:pmu_labworks/domain/models
/card.dart';
import
'package:pmu_labworks/domain/models
/home.dart';
import
'package:pmu_labworks/domain/models
/character.dart';

extension CharactersDtoToModel on
CharactersDto {
  HomeData toDomain() => HomeData(
    data: data?.map((e) =>
e.toDomain()).toList(),
    nextPage:
meta?.pagination?.next,
  );
}

extension CharacterDataDtoToModel
on CharacterDataDto {
  CardData toDomain() => CardData(
    id: id,
    title: type ?? 'UNKNOWN',
    text:
_makeDescriptionText(attributes?.bo
rn, attributes?.died),
    character:
CharacterData(name:
attributes?.name ?? 'Noname',
avatarUrl: attributes?.image),
  );

  String
_makeDescriptionText(String? born,
String? died) {
    if (born != null && died !=
null) {
      return '$born - $died';
    } else if (born != null) {
      return 'born: $born';
    } else if (died != null) {
      return 'died: $died';
    } else {
      return '';
    }
  }
}
import
'package:pmu_labworks/domain/models
/home.dart';

typedef OnErrorCallback = void

```

```

Function(String? error);

abstract class ApiInterface {
  Future<HomeData?>
loadData({OnErrorCallback?
onError});
}
import
'package:pmu_labworks/data/reposito
ries/api_interface.dart';
import
'package:pmu_labworks/domain/models
/card.dart';
import
'package:pmu_labworks/domain/models
/home.dart';
import
'package:pmu_labworks/domain/models
/character.dart';

class MockRepository extends
ApiInterface {
  @override
  Future<HomeData?>
loadData({OnErrorCallback?
onError}) async {
    final data = [
      CardData(
        title: 'Oh my god',
        text: 'this app is so cool
' * 10,
        character: CharacterData(
          name: 'Steve',
          avatarUrl:
'https://preview.free3d.com/img/201
6/03/1875481443430303321/hld8c0oa.j
pg',
        ),
      ),
      CardData(
        title: 'Listen to me',
        text: 'This app is like
Half-Life 3 - it will never be
released',
        character: CharacterData(
          name: 'G-Man',
          avatarUrl:
'https://us-tuna-
sounds-
images.voicemod.net/356aeabd-18d5-
40d8-af31-b479f4eff183-
1704672174928.png',
        ),
      ),
      CardData(
        title: 'BREAKING!!!',
        text: 'ONLY NOW you can
purchase the ability to put likes
and dislikes for only 299$',
        character: CharacterData(
          name: 'Bethesda',
          avatarUrl:
'https://i.playground.ru/p/BWixorST
eZQfoPvdVL9lgA.jpeg',
        ),
      ),
    ],
  );
}

```

```

        CardData(
          title: 'Test',
          text: 'Test',
          character: CharacterData(
            name: 'Noname',
          ),
        ),
        CardData(
          title: 'Test',
          text: 'Test',
          character: CharacterData(
            name: 'Noname',
          ),
        ),
      ];

      return HomeData(data: data);
    }
  }
import 'package:dio/dio.dart';
import
'package:pmu_labworks/data/dtos/characters_dto.dart';
import
'package:pmu_labworks/data/mappers/characters_mapper.dart';
import
'package:pmu_labworks/data/repositories/api_interface.dart';
import
'package:pmu_labworks/domain/models/home.dart';
import
'package:pretty_dio_logger/pretty_dio_logger.dart';

class PotterDBRepository extends ApiInterface {
  static final Dio _dio = Dio()

  ..interceptors.add(PrettyDioLogger(
    requestHeader: true,
    requestBody: true,
  ));

  static const String _baseUrl =
'https://api.potterdb.com';

  @override
Future<HomeData?> loadData({
  OnErrorCallback? onError,
  String? q,
  int page = 1,
  int pageSize = 25,
}) async {
  try {
    const String url =
'$_baseUrl/v1/characters';

    final Response<dynamic>
response = await
_dio.get<Map<dynamic, dynamic>>(
  url,
  queryParameters: {
    'filter[name_cont]': q,
    'page[number]': page,

```

```

    'page[size]': pageSize,
  },
);

    final CharactersDto dto =
CharactersDto.fromJson(response.data
as Map<String, dynamic>);
    final HomeData data =
dto.toDomain();
    return data;
  } on DioException catch (e) {
onError?.call(e.error?.toString());
    return null;
  }
}
import
'package:flutter/widgets.dart';
import
'package:flutter_svg/flutter_svg.dart';
import
'package:pmu_labworks/components/resources.g.dart';

abstract class SvgObjects {
  static void init() {
    final pics = <String>[
      R.ASSETS_SVG_RU_SVG,
      R.ASSETS_SVG_UK_SVG,
    ];
    for (final String p in pics) {
      final loader =
SvgAssetLoader(p);

      svg.cache.putIfAbsent(loader.cacheKey(
null), () =>
loader.loadBytes(null));
    }
  }
}

class SvgRu extends StatelessWidget {
  const SvgRu({super.key});

  @override
Widget build(BuildContext
context) {
    return
SvgPicture.asset(R.ASSETS_SVG_RU_SVG);
  }
}

class SvgUk extends StatelessWidget {
  const SvgUk({super.key});

  @override
Widget build(BuildContext
context) {
    return
SvgPicture.asset(R.ASSETS_SVG_UK_SVG);
  }
}

```

```

    }
  }
import
'package:flutter/material.dart';
import
'package:pmu_labworks/components/extensions/context_x.dart';
import
'package:pmu_labworks/domain/models/card.dart';

class DetailsPage extends
StatelessWidget {
  final CardData model;

  const DetailsPage(this.model,
{super.key});

  @override
  Widget build(BuildContext
context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          context.locale.comment(model.character.name),
        ),
      ),
      body: Padding(
        padding: const
EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment:
CrossAxisAlignment.start,
          children: [
            Stack(
              children: [
                ClipRRect(
                  borderRadius:
BorderRadius.circular(8.0),
                  child:
model.character.avatarUrl != null
?
Image.network(
model.character.avatarUrl!,
height:
150,
width:
150,
fit:
BoxFit.cover,
)
: Container(
height:
150,
width:
150,
color:
Colors.grey,
child:
const Icon(Icons.person, size: 50),
),
            Positioned(

```

```

              left: 8,
              bottom: 8,
              child: Container(
                padding: const
EdgeInsets.symmetric(vertical: 4.0,
horizontal: 8.0),
                color:
Colors.black.withOpacity(0.6),
                child: Text(
                  _shortenNickname(model.character.name, maxLength: 13),
                ),
                style: const
TextStyle(
                  color:
Colors.white,
                  fontWeight:
FontWeight.bold,
                  fontSize:
16,
                ),
              ),
            ),
          ],
        ),
        const SizedBox(height:
24),
        Text(
          model.title,
          style: const
TextStyle(
            fontSize: 24,
            fontWeight:
FontWeight.bold,
          ),
        ),
        const SizedBox(height:
16),
        Text(
          model.text,
          style: const
TextStyle(
            fontSize: 16,
          ),
        ),
      ],
    ),
  );
}

String _shortenNickname(String
nickname, {int maxLength = 15}) {
  if (nickname.length >
maxLength) {
    return nickname.substring(0,
maxLength) + '...';
  }
  return nickname;
}
import
'package:flutter/material.dart';

class ErrorDialog extends

```

```

StatelessWidget {
  final String? error;

  const ErrorDialog(this.error,
    {super.key});

  @override
  Widget build(BuildContext
context) {
    return Center(
      child: Material(
        color: Colors.transparent,
        child: Container(
          margin: const
EdgeInsets.all(36),
          padding: const
EdgeInsets.all(20),
          decoration: const
BoxDecoration(
            color: Colors.grey,
            borderRadius:
BorderRadius.all(Radius.circular(10
)),
          ),
          child: Column(
            mainAxisAlignment:
MainAxisSize.min,
            children: [
              const
Icon(Icons.error, color:
Colors.white),
              const
SizedBox(height: 12),
              Text(
                error ?? 'UNKNOWN',
                style:
Theme.of(context).textTheme.bodyLar
ge?.copyWith(color: Colors.white),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
import
'package:flutter/material.dart';
import
'package:pmu_labworks/view/dialogs/
error_dialog.dart';

void showErrorDialog(
  BuildContext context, {
    required String? error,
  }) {
  showDialog(
    context: context,
    builder: (_) =>
ErrorDialog(error),
  );
}
import
'package:copy_with_extension/copy_w
ith_extension.dart';
import

```

```

'package:equatable/equatable.dart';
import
'package:pmu_labworks/domain/models
/home.dart';

part 'state.g.dart';

@CopyWith()
class HomeState extends Equatable {
  final HomeData? data;
  final bool isLoading;
  final bool isPaginationLoading;
  final String? error;

  const HomeState({
    this.data,
    this.isLoading = false,
    this.isPaginationLoading =
false,
    this.error,
  });

  @override
  List<Object?> get props => [
    data,
    isLoading,
    isPaginationLoading,
    error,
  ];
}
// GENERATED CODE - DO NOT MODIFY
BY HAND

part of 'state.dart';

//
*****
*****
****
// CopyWithGenerator
//
*****
*****
*****

abstract class _$HomeStateCWProxy {
  HomeState data(HomeData? data);

  HomeState isLoading(bool
isLoading);

  HomeState
isPaginationLoading(bool
isPaginationLoading);

  HomeState error(String? error);

  /// This function **does
support** nullification of nullable
fields. All `null` values passed to
`non-nullable` fields will be
ignored. You can also use
`HomeState(...).copyWith.fieldName(
...)` to override fields one at a
time with nullification support.
  ///

```

```

    /// Usage
    /// ```dart
    /// HomeState(...).copyWith(id:
    12, name: "My name")
    /// ```
    HomeState call({
      HomeData? data,
      bool? isLoading,
      bool? isPaginationLoading,
      String? error,
    });
  }

  /// Proxy class for `copyWith`
  /// functionality. This is a callable
  /// class and can be used as follows:
  /// `instanceOfHomeState.copyWith(...)`
  /// . Additionally contains functions
  /// for specific fields e.g.
  /// `instanceOfHomeState.copyWith.field
  /// Name(...)`
  class _$HomeStateCWProxyImpl
  implements _$HomeStateCWProxy {
    const
    _$HomeStateCWProxyImpl(this._value)
    ;

    final HomeState _value;

    @override
    HomeState data(HomeData? data) =>
    this(data: data);

    @override
    HomeState isLoading(bool
    isLoading) => this(isLoading:
    isLoading);

    @override
    HomeState
    isPaginationLoading(bool
    isPaginationLoading) =>
    this(isPaginationLoading:
    isPaginationLoading);

    @override
    HomeState error(String? error) =>
    this(error: error);

    @override

    /// This function does
    support nullable fields. All `null` values passed to
    `non-nullable` fields will be
    ignored. You can also use
    `HomeState(...).copyWith.fieldName(
    ...)` to override fields one at a
    time with nullification support.
    ///
    /// Usage
    /// ```dart
    /// HomeState(...).copyWith(id:
    12, name: "My name")
    /// ```
    HomeState call({

```

```

    Object? data = const
    $CopyWithPlaceholder(),
    Object? isLoading = const
    $CopyWithPlaceholder(),
    Object? isPaginationLoading =
    const $CopyWithPlaceholder(),
    Object? error = const
    $CopyWithPlaceholder(),
  }) {
    return HomeState(
      data: data == const
      $CopyWithPlaceholder()
      ? _value.data
      // ignore:
      cast_nullable_to_non_nullable
      : data as HomeData?,
      isLoading: isLoading == const
      $CopyWithPlaceholder() || isLoading
      == null
      ? _value.isLoading
      // ignore:
      cast_nullable_to_non_nullable
      : isLoading as bool,
      isPaginationLoading:
      isPaginationLoading ==
      const $CopyWithPlaceholder() ||
      isPaginationLoading == null
      ?
      _value.isPaginationLoading
      // ignore:
      cast_nullable_to_non_nullable
      : isPaginationLoading
      as bool,
      error: error == const
      $CopyWithPlaceholder()
      ? _value.error
      // ignore:
      cast_nullable_to_non_nullable
      : error as String?,
    );
  }

  extension $HomeStateCopyWith on
  HomeState {
    /// Returns a callable class that
    can be used as follows:
    `instanceOfHomeState.copyWith(...)`
    or like
    so: `instanceOfHomeState.copyWith.fi
    eldName(...)`
    // ignore:
    library_private_types_in_public_api
    _$HomeStateCWProxy get copyWith
    => _$HomeStateCWProxyImpl(this);
  }

  abstract class HomeEvent {
    const HomeEvent();
  }

  class HomeLoadDataEvent extends
  HomeEvent {
    final String? search;
    final int? nextPage;

    const

```

```

HomeLoadDataEvent({this.search,
this.nextPage});
}
import
'package:flutter_bloc/flutter_bloc.
dart';
import
'package:pmu_labworks/data/reposito
ries/potterdb_repository.dart';
import
'package:pmu_labworks/view/home_pag
e/bloc/events.dart';
import
'package:pmu_labworks/view/home_pag
e/bloc/state.dart';

class HomeBloc extends
Bloc<HomeEvent, HomeState> {
  final PotterDBRepository repo;

  HomeBloc(this.repo) : super(const
HomeState()) {

on<HomeLoadDataEvent>(_onLoadData);
  }

  Future<void>
_onLoadData(HomeLoadDataEvent
event, Emitter<HomeState> emit)
async {
    if (event.nextPage == null) {

emit(state.copyWith(isLoading:
true));
    } else {

emit(state.copyWith(isPaginationLoa
ding: true));
    }

    String? error;

    final data = await
repo.loadData(
      q: event.search,
      page: event.nextPage ?? 1,
      onError: (e) => error = e,
    );

    if (event.nextPage != null) {
      data?.data?.insertAll(0,
state.data?.data ?? []);
    }

    emit(state.copyWith(
      isLoading: false,
      isPaginationLoading: false,
      data: data,
      error: error,
    ));
  }
}
part of 'home_page.dart';

typedef OnReactionCallback = void
Function(

```

```

  String? id, String nickname,
  bool isLiked, bool isDisliked)?;

class _Card extends StatelessWidget
{
  final String? id;
  final String title;
  final String text;
  final CharacterData character;
  final bool isLiked;
  final bool isDisliked;
  final OnReactionCallback? onLike;
  final OnReactionCallback?
onDislike;
  final VoidCallback? onTap;

  const _Card({
    this.id,
    required this.title,
    required this.text,
    required this.character,
    this.isLiked = false,
    this.isDisliked = false,
    this.onLike,
    this.onDislike,
    this.onTap,
  });

  factory _Card.fromData(
    CardData data, {
    bool isLiked = false,
    bool isDisliked = false,
    OnReactionCallback? onLike,
    OnReactionCallback? onDislike,
    VoidCallback? onTap,
  }) =>
    _Card(
      id: data.id,
      title: data.title,
      text: data.text,
      character: data.character,
      isLiked: isLiked,
      isDisliked: isDisliked,
      onLike: onLike,
      onDislike: onDislike,
      onTap: onTap,
    );

  void _onLike() {
    onLike?.call(id,
character.name, isLiked,
isDisliked);
  }

  void _onDislike() {
    onDislike?.call(id,
character.name, isLiked,
isDisliked);
  }

  @override
  Widget build(BuildContext
context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(

```

```

        margin: const
EdgeInsets.all(16),
padding: const
EdgeInsets.all(16),
decoration: BoxDecoration(
  color: Colors.white70,
  borderRadius:
BorderRadius.circular(16),
border: Border.all(color:
Colors.grey),
boxShadow: [
  BoxShadow(
    color:
Colors.grey.withOpacity(.5),
    spreadRadius: 4,
    offset: const
Offset(0, 5),
    blurRadius: 8,
  ),
],
),
child: Row(
  crossAxisAlignment:
CrossAxisAlignment.start,
  children: [
    Column(
      children: [
        ClipRRect(
          borderRadius:
BorderRadius.circular(30),
          child: SizedBox(
            height: 80,
            width: 80,
            child:
character.avatarUrl != null &&
character.avatarUrl!.isNotEmpty
?
Image.network(
character.avatarUrl!,
fit:
BoxFit.cover,
):
Container(
height:
80,
width:
80,
color:
Colors.grey,
child:
const Icon(Icons.person, size: 40),
),
),
const
SizedBox(height: 8),
SizedBox(
width: 80,
child: Text(
character.name,
style:
Theme.of(context).textTheme.bodyMed
ium,
maxLines: 1,

```

```

overflow:
TextOverflow.ellipsis,
textAlign:
TextAlign.center,
),
),
Row(
  children: [
    IconButton(
      iconSize: 20,
      icon: Icon(
        isLiked ?
Icons.thumb_up :
Icons.thumb_up_outlined,
color:
isLiked ? Colors.blue :
Colors.grey,
),
onPressed:
_onLike,
),
    IconButton(
      iconSize: 20,
      icon: Icon(
        isDisliked
?
Icons.thumb_down
:
Icons.thumb_down_outlined,
color:
isDisliked ? Colors.red :
Colors.grey,
),
onPressed:
_onDislike,
),
],
),
Expanded(
  child: Padding(
    padding: const
EdgeInsets.only(left: 16.0),
    child: Column(
      crossAxisAlignment:
CrossAxisAlignment.start,
      children: [
        Text(
          title,
          style:
Theme.of(context).textTheme.headlin
eSmall,
),
const
SizedBox(height: 8),
Text(
text,
style:
Theme.of(context).textTheme.bodyLar
ge,
),
],
),
),

```

```

    ),
  ],
),
);
}
}
import 'dart:math';

import
'package:flutter/cupertino.dart';
import
'package:flutter/material.dart';
import
'package:flutter_bloc/flutter_bloc.
dart';
import
'package:pmu_labworks/components/ex
tensions/context_x.dart';
import
'package:pmu_labworks/components/ut
ils/debounce.dart';
import
'package:pmu_labworks/domain/models
/card.dart';
import
'package:pmu_labworks/domain/models
/character.dart';
import
'package:pmu_labworks/view/common/s
vg_objects.dart';
import
'package:pmu_labworks/view/details_
page/details_page.dart';
import
'package:pmu_labworks/view/home_pag
e/bloc/bloc.dart';
import
'package:pmu_labworks/view/home_pag
e/bloc/events.dart';
import
'package:pmu_labworks/view/home_pag
e/bloc/state.dart';
import
'package:pmu_labworks/view/reaction
_bloc/reaction_bloc.dart';
import
'package:pmu_labworks/view/reaction
_bloc/reaction_event.dart';
import
'package:pmu_labworks/view/reaction
_bloc/reaction_state.dart';
import
'package:pmu_labworks/view/locale_b
loc/locale_bloc.dart';
import
'package:pmu_labworks/view/locale_b
loc/locale_events.dart';
import
'package:pmu_labworks/view/locale_b
loc/locale_state.dart';

part 'card.dart';

class HomePage extends
StatefulWidget {

```

```

  const HomePage({super.key,
required this.title});

  final String title;

  @override
  State<HomePage> createState() =>
  _HomePageState();
}

class _HomePageState extends
State<HomePage> {
  Color _color =
Colors.transparent;

  @override
  void initState() {
    super.initState();
    _color = _generateColor();
  }

  @override
  Widget build(BuildContext
context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: _color,
        title: Column(
          crossAxisAlignment:
CrossAxisAlignment.start,
          children: [
            Text(widget.title,
              style: const
TextStyle(
                fontSize: 20,
                fontWeight:
FontWeight.bold,
              )),
            Text(
              context.locale.credits + '
Factorino',
                style: TextStyle(
                  fontSize: 12,
                  fontStyle:
FontStyle.italic,
                ),
              ),
          ],
        ),
      ),
      body: const _Body(),
    );
  }

  Color _generateColor() {
    final Random random = Random();
    final int red =
(random.nextInt(106) + 150);
    final int green =
(random.nextInt(106) + 150);
    final int blue =
(random.nextInt(106) + 150);

    return Color.fromARGB(255, red,
green, blue);
  }
}

```

```

    }
}

class _Body extends StatefulWidget {
  const _Body();

  @override
  State<_Body> createState() =>
    _BodyState();
}

class _BodyState extends
  State<_Body> {
  final searchController =
    TextEditingController();
  final scrollController =
    ScrollController();

  @override
  void initState() {
    SvgObjects.init();

    WidgetsBinding.instance.addPostFrame
    eCallback(() {

      context.read<HomeBloc>().add(const
      HomeLoadDataEvent());

      context.read<ReactionBloc>().add(const
      LoadReactionsEvent());
    });

    scrollController.addListener(_onNext
    PageListener);

    super.initState();
  }

  @override
  void dispose() {
    searchController.dispose();
    scrollController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext
  context) {
    return Padding(
      padding: EdgeInsets.only(top:
      MediaQuery.of(context).padding.top)
      ,
      child: Column(
        children: [
          Row(
            children: [
              Expanded(
                flex: 4,
                child: Padding(
                  padding: const
                  EdgeInsets.all(12),
                  child:
                  CupertinoSearchTextField(

```

```

        controller:
        searchController,
        placeholder:
        context.locale.search,
        onChanged:
        (search) {
          Debounce.run(
            () =>
            context.read<HomeBloc>().add(HomeLo
            adDataEvent(search: search))),
          ),
        ),
        ),
        GestureDetector(
          onTap: () =>
          context.read<LocaleBloc>().add(const
          t ChangeLocaleEvent()),
          child:
          SizedBox.square(
            dimension: 50,
            child: Padding(
              padding: const
              EdgeInsets.only(right: 12),
              child:
              BlocBuilder<LocaleBloc,
              LocaleState>(
                builder:
                (context, state) {
                  return
                  state.currentLocale.languageCode ==
                  'ru'
                  ? const
                  SvgRu()
                  : const
                  SvgUk();
                },
              ),
            ),
          ),
        ],
      ),
      BlocBuilder<HomeBloc,
      HomeState>(
        builder: (context,
        state) => state.error != null
        ? Text(
          state.error ??
          '',
          style:
          Theme.of(context).textTheme.headlin
          eSmall?.copyWith(color:
          Colors.red),
        )
        : state.isLoading
        ? const
        CircularProgressIndicator()
        :
        BlocBuilder<ReactionBloc,
        ReactionState>(
          builder:
          (context, reactionState) {
            return
            Expanded(
              child:

```



```

        print('$isLiked $isDisliked = $action');

ScaffoldMessenger.of(context).showSnackBar(SnackBar(
  content: Text(

context.locale.comment(nickname) +
' ' + action,
    style:
Theme.of(context).textTheme.bodyLarge,
    ),
    backgroundColor:
Colors.white70,
    duration: const
Duration(seconds: 1),
    ));
  });
}

void _onNextPageListener() {
  if (scrollController.offset >=
scrollController.position.maxScrollExtent) {
    // preventing multiple
pagination request on multiple
swipes
    final bloc =
context.read<HomeBloc>();
    if
(!bloc.state.isPaginationLoading) {
      bloc.add(HomeLoadDataEvent(
        search:
searchController.text,
        nextPage:
bloc.state.data?.nextPage,
      ));
    }
  }
}
import
'package:copy_with_extension/copy_with_extension.dart';
import
'package:equatable/equatable.dart';
import
'package:flutter/material.dart';

part 'locale_state.g.dart';

@CopyWith()
class LocaleState extends Equatable {
  final Locale currentLocale;

  const LocaleState({required
this.currentLocale});

  @override
  List<Object?> get props =>
[currentLocale];
}

```

```

// GENERATED CODE - DO NOT MODIFY BY HAND

part of 'locale_state.dart';

//
*****
*****
****
// CopyWithGenerator
//
*****
*****
****

abstract class _$LocaleStateCWProxy {
  LocaleState currentLocale(Locale currentLocale);

  /// This function **does support** nullification of nullable fields. All `null` values passed to `non-nullable` fields will be ignored. You can also use `LocaleState(...).copyWith.fieldName(...)` to override fields one at a time with nullification support.
  ///
  /// Usage
  /// ```dart
  /// LocaleState(...).copyWith(id: 12, name: "My name")
  /// ```
  LocaleState call({
    Locale? currentLocale,
  });
}

/// Proxy class for `copyWith` functionality. This is a callable class and can be used as follows: `instanceOfLocaleState.copyWith(...)` . Additionally contains functions for specific fields e.g. `instanceOfLocaleState.copyWith.fieldName(...)`
class _$LocaleStateCWProxyImpl implements _$LocaleStateCWProxy {
  const
  _$LocaleStateCWProxyImpl(this._value);

  final LocaleState _value;

  @override
  LocaleState currentLocale(Locale currentLocale) =>
this(currentLocale: currentLocale);

  @override

  /// This function **does support** nullification of nullable fields. All `null` values passed to `non-nullable` fields will be

```

```

ignored. You can also use
`LocaleState(...).copyWith.fieldName(...)` to override fields one at a
time with nullification support.
///
/// Usage
/// ```dart
/// LocaleState(...).copyWith(id:
12, name: "My name")
/// ```
LocaleState call({
  Object? currentLocale = const
$CopyWithPlaceholder(),
}) {
  return LocaleState(
    currentLocale: currentLocale
== const $CopyWithPlaceholder() ||
currentLocale == null
? _value.currentLocale
// ignore:
cast_nullable_to_non_nullable
: currentLocale as
Locale,
  );
}

extension $LocaleStateCopyWith on
LocaleState {
  /// Returns a callable class that
  can be used as follows:
  `instanceOfLocaleState.copyWith(...)` or like
  so: `instanceOfLocaleState.copyWith.
fieldName(...)`
  // ignore:
library_private_types_in_public_api
_$LocaleStateCWProxy get copyWith
=> _$LocaleStateCWProxyImpl(this);
}
abstract class LocaleEvent {
  const LocaleEvent();
}

class ChangeLocaleEvent extends
LocaleEvent {
  const ChangeLocaleEvent();
}
import
'package:flutter/material.dart';
import
'package:flutter_bloc/flutter_bloc.
dart';
import
'package:pmu_labworks/components/lo
cale/l10n/app_locale.dart';
import
'package:pmu_labworks/view/locale_b
loc/locale_events.dart';
import
'package:pmu_labworks/view/locale_b
loc/locale_state.dart';

class LocaleBloc extends
Bloc<LocaleEvent, LocaleState> {
  LocaleBloc(Locale defaultLocale)

```

```

: super(LocaleState(currentLocale:
defaultLocale)) {

on<ChangeLocaleEvent>(_onChangeLoca
le);
}

Future<void>
_onChangeLocale(ChangeLocaleEvent
event, Emitter<LocaleState> emit)
async {
  final toChange =
AppLocale.supportedLocales
.firstWhere((e) =>
e.languageCode !=
state.currentLocale.languageCode);

emit(state.copyWith(currentLocale:
toChange));
}
import
'package:copy_with_extension/copy_w
ith_extension.dart';
import
'package:equatable/equatable.dart';

part 'reaction_state.g.dart';

@CopyWith()
class ReactionState extends
Equatable {
  final List<String>? likedIds;
  final List<String>? dislikedIds;

  const ReactionState({required
this.likedIds, required
this.dislikedIds});

  @override
  List<Object?> get props =>
[likedIds, dislikedIds];
}
// GENERATED CODE - DO NOT MODIFY
BY HAND

part of 'reaction_state.dart';

//
*****
*****
****
// CopyWithGenerator
//
*****
*****
****

abstract class
_$ReactionStateCWProxy {
  ReactionState
likedIds(List<String>? likedIds);

  ReactionState
dislikedIds(List<String>?
dislikedIds);
}

```

```

    /// This function **does
    support** nullification of nullable
    fields. All `null` values passed to
    `non-nullable` fields will be
    ignored. You can also use
    `ReactionState(...).copyWith.fieldName(...)` to override fields one at
    a time with nullification support.
    ///
    /// Usage
    /// ```dart
    ///
    ReactionState(...).copyWith(id: 12,
    name: "My name")
    /// ```
    ReactionState call({
      List<String>? likedIds,
      List<String>? dislikedIds,
    });
  }

  /// Proxy class for `copyWith`
  functionality. This is a callable
  class and can be used as follows:
  `instanceOfReactionState.copyWith(...)`
  .Additionally contains
  functions for specific fields e.g.
  `instanceOfReactionState.copyWith.fieldName(...)`
  class _$ReactionStateCWProxyImpl
  implements _$ReactionStateCWProxy {
    const
    _$ReactionStateCWProxyImpl(this._value);

    final ReactionState _value;

    @override
    ReactionState
    likedIds(List<String>? likedIds) =>
    this(likedIds: likedIds);

    @override
    ReactionState
    dislikedIds(List<String>?
    dislikedIds) => this(dislikedIds:
    dislikedIds);

    @override

    /// This function **does
    support** nullification of nullable
    fields. All `null` values passed to
    `non-nullable` fields will be
    ignored. You can also use
    `ReactionState(...).copyWith.fieldName(...)` to override fields one at
    a time with nullification support.
    ///
    /// Usage
    /// ```dart
    ///
    ReactionState(...).copyWith(id: 12,
    name: "My name")
    /// ```

```

```

    ReactionState call({
      Object? likedIds = const
      $CopyWithPlaceholder(),
      Object? dislikedIds = const
      $CopyWithPlaceholder(),
    }) {
      return ReactionState(
        likedIds: likedIds == const
        $CopyWithPlaceholder()
        ? _value.likedIds
        // ignore:
        cast_nullable_to_non_nullable
        : likedIds as
        List<String>?,
        dislikedIds: dislikedIds ==
        const $CopyWithPlaceholder()
        ? _value.dislikedIds
        // ignore:
        cast_nullable_to_non_nullable
        : dislikedIds as
        List<String>?,
      );
    }
  }

  extension $ReactionStateCopyWith on
  ReactionState {
    /// Returns a callable class that
    can be used as follows:
    `instanceOfReactionState.copyWith(...)`
    or like
    so: `instanceOfReactionState.copyWith.h.fieldName(...)`
    // ignore:
    library_private_types_in_public_api
    _$ReactionStateCWProxy get
    copyWith =>
    _$ReactionStateCWProxyImpl(this);
  }

  abstract class ReactionEvent {
    const ReactionEvent();
  }

  class LoadReactionsEvent extends
  ReactionEvent {
    const LoadReactionsEvent();
  }

  class ChangeLikeEvent extends
  ReactionEvent {
    final String id;

    const ChangeLikeEvent(this.id);
  }

  class ChangeDislikeEvent extends
  ReactionEvent {
    final String id;

    const
    ChangeDislikeEvent(this.id);
  }
  import
  'package:flutter_bloc/flutter_bloc.dart';
  import

```

```

'package:shared_preferences/shared_preferences.dart';
import 'reaction_event.dart';
import 'reaction_state.dart';

const String _likedPrefsKey =
'liked';
const String _dislikedPrefsKey =
'disliked';

class ReactionBloc extends
Bloc<ReactionEvent, ReactionState>
{
  ReactionBloc() : super(const
ReactionState(likedIds: [],
dislikedIds: [])) {

on<ChangeLikeEvent>(_onChangeLike);

on<ChangeDislikeEvent>(_onChangeDis
like);

on<LoadReactionsEvent>(_onLoadReact
ions);
  }

  Future<void>
_onLoadReactions(LoadReactionsEvent
event, Emitter<ReactionState> emit)
async {
    final prefs = await
SharedPreferences.getInstance();
    final likedData =
prefs.getStringList(_likedPrefsKey)
;
    final dislikedData =
prefs.getStringList(_dislikedPrefsK
ey);

    emit(state.copyWith(
      likedIds: likedData ?? [],
      dislikedIds: dislikedData ??
[],
    ));
  }

  Future<void>
_onChangeLike(ChangeLikeEvent
event, Emitter<ReactionState> emit)
async {
    final updatedLikedList =
List<String>.from(state.likedIds ??
[]);
    final updatedDislikedList =
List<String>.from(state.dislikedIds
?? []);

    if
(updatedLikedList.contains(event.id
)) {
updatedLikedList.remove(event.id);
    } else {
updatedLikedList.add(event.id);
  }
}

```

```

updatedDislikedList.remove(event.id
); // Убираем дизлайк, если он был
  }

    final prefs = await
SharedPreferences.getInstance();

prefs.setStringList(_likedPrefsKey,
updatedLikedList);

prefs.setStringList(_dislikedPrefsK
ey, updatedDislikedList);

    emit(state.copyWith(
      likedIds: updatedLikedList,
      dislikedIds:
updatedDislikedList,
    ));
  }

  Future<void>
_onChangeDislike(ChangeDislikeEvent
event, Emitter<ReactionState> emit)
async {
    final updatedLikedList =
List<String>.from(state.likedIds ??
[]);
    final updatedDislikedList =
List<String>.from(state.dislikedIds
?? []);

    if
(updatedDislikedList.contains(event
.id)) {
updatedDislikedList.remove(event.id
);
    } else {
updatedDislikedList.add(event.id);
updatedLikedList.remove(event.id);
// Убираем лайк, если он был
    }

    final prefs = await
SharedPreferences.getInstance();

prefs.setStringList(_likedPrefsKey,
updatedLikedList);

prefs.setStringList(_dislikedPrefsK
ey, updatedDislikedList);

    emit(state.copyWith(
      likedIds: updatedLikedList,
      dislikedIds:
updatedDislikedList,
    ));
  }
}
import 'dart:io';

import
'package:flutter/material.dart';
import

```

```

'package:flutter_bloc/flutter_bloc.
dart';
import
'package:pmu_labworks/components/lo
cale/l10n/app_locale.dart';
import
'package:pmu_labworks/data/reposito
ries/potterdb_repository.dart';
import
'package:pmu_labworks/view/home_pag
e/bloc/bloc.dart';
import
'package:pmu_labworks/view/home_pag
e/home_page.dart';
import
'package:pmu_labworks/view/reaction
_bloc/reaction_bloc.dart';
import
'package:pmu_labworks/view/locale_b
loc/locale_bloc.dart';
import
'package:pmu_labworks/view/locale_b
loc/locale_state.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget
{
  const MyApp({super.key});

  @override
  Widget build(BuildContext
context) {
    return
BlocProvider<LocaleBloc>(
  lazy: false,
  create: (context) =>
LocaleBloc(Locale(Platform.localeNa
me)),
  child:
BlocBuilder<LocaleBloc,
LocaleState>(
  builder: (context, state) {
    return MaterialApp(
      title: "Comments App",
      locale:
state.currentLocale,
      localizationsDelegates:
AppLocale.localizationsDelegates,
      supportedLocales:
AppLocale.supportedLocales,

      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme:
ColorScheme.fromSeed(seedColor:
Colors.blue),
        useMaterial3: true,
      ),
      home:
RepositoryProvider<PotterDBReposito
ry>(
        lazy: true,
        create: (_) =>

```

```

PotterDBRepository(),
        child:
BlocProvider<ReactionBloc>(
          lazy: false,
          create: (context)
=> ReactionBloc(),
          child:
BlocProvider<HomeBloc>(
            lazy: false,
            create: (context)
=>
HomeBloc(context.read<PotterDBRepos
itory>()),
            child: const
HomePage(title: "Comments App"),
          ),
        ),
      ),
    ),
  ),
);
}
}

```